



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

Μάθημα: Γραμμική και Συνδυαστική Βελτιστοποίηση

Τελικό Project

Ον/μο: Σάλιαρης Γεώργιος

ΑΜ: 1092584

Υπεύθυνη καθηγήτρια: Δασκαλάκη Σοφία

Ακαδημαϊκό έτος: 2024-25

Δομή της εργασίας

- 1. Παρουσίαση προβλήματος**
- 2. Παρουσίαση βασικών εννοιών σχετικές με το πρόβλημα**
- 3. Μοντελοποίηση του προβλήματος και επίλυση με τη μέθοδο Simplex**
- 4. Δυϊκό πρόβλημα**
- 5. Σχολιασμός λύσεων πρωτεύοντος και δυϊκού προβλήματος**
- 6. Επίλυση ενός πιο περίπλοκου**
- 7. Σύγκριση/σχολιασμός των δύο τύπων προβλημάτων (απλού και περίπλοκου)**
- 8. Πιθανές τροποποιήσεις/διαφορετικές τεχνικές επίλυσης στα δύο προβλήματα**
- 9. Συμπεράσματα-Γενικός σχολιασμός**
- 10. Κώδικας**
- 11. Βιβλιογραφία**

1.Παρουσίαση προβλήματος

Στόχος της παρούσης εργασίας είναι ο σχεδιασμός ενός μηνιαίου προγράμματος γυμναστικής με τη μέθοδο *Simplex* έτσι ώστε να μεγιστοποιήσουμε ένα *fitness score* διατηρώντας παράλληλα το πλάνο μας (σχετικά) ισορροπημένο, ασφαλές και κατά το δυνατόν ρεαλιστικό. Αρχικά μοντελοποιούμε το πρόβλημα (χωρίς να μας νοιάζει αν βρίσκεται σε τυπική μορφή) ως ένα πρόβλημα γραμμικού προγραμματισμού (έτσι ώστε να είναι σε θέση να επιλυθεί με τη μέθοδο *Simplex*) και προβαίνουμε στην επίλυσή του, (στην κατασκευή του μηνιαίου προγράμματος δηλαδή). Προχωρώντας, αφού πάλι έχουμε λύσει το δυαδικό πρόβλημα, όπως επίσης και ένα μεγαλύτερης πολυπλοκότητας πρόβλημα, αντιπαραβάλλουμε τα δύο από άποψη απαιτούμενου χρόνου, πόρων του υπολογιστή κλπ. Τέλος, γίνεται μία συζήτηση για πιθανές τροποποιήσεις του προβλήματος, διαφορετικές τεχνικές επίλυσης και μοντελοποίησης όπου εξετάζουμε το εν λόγω πρόβλημα όχι από την σκοπιά της μεθόδου *Simplex*. Η εργασία κλείνει με την παρουσίαση των συμπερασμάτων και της σχετικής βιβλιογραφίας η οποία χρησιμοποιήθηκε για την εκπόνησή της.

2. Παρουσίαση βασικών εννοιών σχετικές με το πρόβλημα

-*Fitness score*: Πρόκειται για έναν δείκτη οποίος συγκεντρώνει τα οφέλη από το κάθε είδος προπόνησης (αντοχή, δύναμη κλπ) σε ένα νούμερο. Στο μοντέλο μας πρόκειται για το άθροισμα των ωρών κάθε τύπου άσκησης επί τους αντίστοιχους πόντους ανά ώρα.

-*Cardio* (αερόβια άσκηση): Κίνηση η οποία ανεβάζει παλμούς για συνεχόμενα λεπτά (πχ ποδήλατο, τρέξιμο, κλπ) με τη μέτρια ένταση να σημαίνει ότι μπορεί κάποιος να μιλήσει και στην αυξημένη μόνο να μπορεί να αρθρώσει μόνο κάποιες λέξεις.

-*HIIT*(*High-Intensity Interval Training*): Προπόνηση η οποία χαρακτηρίζεται από την εναλλαγή διαστημάτων υψηλής έντασης και ήπιας/ξεκούρασης [1].

-*Resistance training*: Προπόνηση με αντιστάσεις/βάρη με στόχο την αύξηση της μυϊκής μάζας του ατόμου.

-*Core & stability training*: Προπόνηση που δυναμώνει τους μύες του κορμού (κοιλιακοί, ραχιαίοι, γλουτοί κ.ά.) ώστε η λεκάνη/σπονδυλική στήλη να μένουν σταθερές ενώ κινούνται τα άκρα.

-*Stretching training*: Διατάσεις είτε ασκήσεις για την ενίσχυση της ευλυγισίας.

-*Mobility training/Yoga*: Προπόνηση η οποία συνδυάζει ήπια δύναμη, ισορροπία, έλεγχο αναπνοής και εύρος κίνησης-θεωρείται χαμηλής/μέτριας έντασης προπόνηση.

-*Plyometrics*: Είναι «εκρηκτικές» ασκήσεις με άλματα/γρήγορες εκκινήσεις. Ο μυς πρώτα τεντώνεται γρήγορα και αμέσως συσπάται δυνατά. Το προηγούμενο αποκαλείται κύκλος διάτασης-βράχυνσης (*Stretch-Shortening Cycle, SSC*) και είναι η βάση της πλειομετρικής προπόνησης [2].

-*Active recovery*: Πολύ χαλαρή κίνηση τις ημέρες ξεκούρασης ύστερα από τις διάφορες προπονήσεις για κυκλοφορία του αίματος και αποκατάσταση των τραυματισμένων μυών.

-*MET*: Είναι μία ποσότητα η οποία μας δείχνει πόση ενέργεια ξοδεύουμε σε μια δραστηριότητα σε σχέση με την απόλυτη ηρεμία. Το 1 *MET* ορίζεται ως η κατανάλωση οξυγόνου σε ηρεμία, δηλαδή 3.5 ml O_2 ανά κιλό σώματος ανά λεπτό ($\sim 1kcal/kg/h$). Έτσι ένα έχουμε μία δραστηριότητα η οποία είναι 5 *MET*, αυτό σημαίνει ότι καίμε μέσω αυτής της δραστηριότητας 5 φορές την ενέργεια που καταναλώνουμε στην απόλυτη ηρεμία [3].

- *$\dot{V}O_2 \max$* (μέγιστη πρόσληψη οξυγόνου): Η μέγιστη ποσότητα οξυγόνου που μπορεί να προσλάβει/χρησιμοποιήσει το σώμα [4].

3.Μοντελοποίηση του προβλήματος και επίλυση με τη μέθοδο Simplex

a) Ορισμός μεταβλητών απόφασης

x_1 : ώρες ανά μήνα για προπόνηση *cardio*

x_2 : ώρες ανά μήνα για προπόνηση *HIIT*

x_3 : ώρες ανά μήνα για προπόνηση ενδυνάμωσης άνω μέρους (πλάτη, ώμοι, χέρια)

x_4 : ώρες ανά μήνα για προπόνηση ενδυνάμωσης κάτω μέρους (πόδια, γλουτοί)

x_5 : ώρες ανά μήνα για προπόνηση *core & stability*

x_6 : ώρες ανά μήνα για προπόνηση *flexibility/stretching*

x_7 : ώρες ανά μήνα για προπόνηση *yoga*

x_8 : ώρες ανά μήνα για προπόνηση *plyometrics*

x_9 : ώρες ανά μήνα για *active recovery*

b) Ορισμός αντικειμενικής συνάρτησης

$$\max Z = 5x_1 + 8x_2 + 6.5x_3 + 6.5x_4 + 4x_5 + 2x_6 + 3x_7 + 5.5x_8 + 1.5x_9 \quad (3.1)$$

Αξίζει στο σημείο αυτό να σταθούμε εδώ λίγο και να εξηγήσουμε κάποια πράγματα για τη μορφή της αντικειμενικής συνάρτησης.

Πρόκειται ουσιαστικά για ένα ενιαίο *fitness score* το οποίο προκύπτει αθροίζοντας τις συνολικές ώρες ανά μήνα για κάθε τύπο άσκησης, με κάθε όρο βέβαια της αντικειμενικής συνάρτησης να έχει έναν συγκεκριμένο συντελεστή. Αυτός ο συντελεστής προκύπτει από τα πόσα *METS* αποδίδει κάθε τύπος άσκησης (μεγαλύτερος συντελεστής σημαίνει ότι η εν λόγω άσκηση είναι πιο έντονη-εξού και στη μεταβλητή x_2 που αντιστοιχεί σε προπόνηση *HIIT* έχει τον υψηλότερο συντελεστή). Επί της ουσίας οι παραπάνω συντελεστές προκύπτουν αν αναλογιστούμε ότι φυσική δραστηριότητα η οποία αντιστοιχεί σε λιγότερο από 3 *METS* χαμηλής έντασης 3 – 5.9 *METS* θεωρείται μέτριας έντασης, και πάνω από 6 *METS* υψηλής έντασης [5]. Επομένως, είναι προφανές ότι πρόκειται για εμπειρικές αλλά σχετικά λογικές τιμές βαρών. Μιας και διαισθητικά σε

ένταση ισχύει: $HIIT > strength > cardio > core > yoga > stretching > active recovery$ (3.2). Υπάρχουν βέβαια και τεχνικές πιο ακριβής ρύθμισης των παραπάνω βαρών αλλά η παρουσίαση των τελευταίων ξεφεύγει από τα πλαίσια της παρούσης εργασίας [6], [7].

c) Περιορισμοί προβλήματος

- 1) *Time-budget* (πόσες ώρες δηλαδή έχουμε διαθέσιμες το μήνα για προπόνηση). Μια υπόθεση εργασίας (αρκετά σκληρή βέβαια) είναι να υποθέσουμε ότι έχουμε 60, δηλαδή ότι ισχύει:

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 \leq 60 \quad (3.3)$$

- 2) *Minimum* αεροβικής προπόνησης, όπου σύμφωνα με το [8] ένας κανονικός ενήλικας χρειάζεται 150 λεπτά γυμναστικής μέτριας έντασης, δηλαδή 10 ώρες το μήνα. Θεωρούμε ως τη βάση (δηλαδή αντιστοιχεί συντελεστής ίσος με τη μονάδα) στη μεταβλητή x_1 , οπότε δεδομένης και της σχέσης (3.2), προκύπτει ο εξής περιορισμός:

$$x_1 + 2x_2 + 0.5x_7 + 1.5x_8 + 0.5x_9 \geq 10 \quad (3.4)$$

- 3) *Minimum* προπόνησης δύναμης (πάλι σύμφωνα με το [8]), θα πρέπει να υπάρχουν 2 (και παραπάνω) μέρες την εβδομάδα και άρα περίπου 8 ώρες το μήνα. Εδώ εστιάζουμε στη δύναμη, άρα η βάση είναι οι μεταβλητές που αντιστοιχούν αυστηρά στις προπονήσεις δύναμης. Ως εκ τούτου, έχουμε τον περιορισμό:

$$0.5x_2 + x_3 + x_4 + 0.5x_5 + 0.5x_8 \geq 8 \quad (3.5)$$

- 4) *Minimum* για προπόνηση ευλιγισίας, σύμφωνα με το [9], το *minimum* είναι 2 – 3 ημέρες την εβδομάδα για αυτό το είδος άσκησης, οπότε χονδρικά έχουμε 6 ώρες το μήνα αποκλειστικά για *flexibility* και *mobility*:

$$x_6 + x_7 \geq 6 \quad (3.6)$$

- 5) Άνω όριο για προπόνηση *HIIT*:

$$x_2 \leq 8 \quad (3.7)$$

Η λογική του παραπάνω περιορισμού προκύπτει από το γεγονός ότι η προπόνηση *HIIT* είναι ιδιαίτερα υψηλής έντασης και καταπονεί ιδιαίτερα τον οργανισμό (άλλωστε ο χρόνος *HIIT* μετράει διπλά, άρα μπορούμε να πιάσουμε τον στόχο μας με λιγότερο χρόνο. Αποτελεί, ένα λογικό μέσο όριο για έναν υγιή ενήλικα. Βλ. [9]

6) Άνω όριο για *plyometrics*:

$$x_8 \leq 6 \quad (3.8)$$

Τα *plyometrics* αξιοποιούν τον κύκλο κίνησης των μυών γνωστό ως SSC [10] και πρόκειται για μια προπόνηση η οποία επιβάλλει υψηλά φορτία σε τένοντες και αρθρώσεις. Γι' αυτό το λόγο, η *NSCA (National Strength and Conditioning Association)* προτείνει προγράμματα με χαμηλή συχνότητα (2-3 προπονήσεις την εβδομάδα) και χαμηλό πλήθος επαναλήψεων (πχ. 3 – 6 σετ, με κάθε σετ να αποτελείται με 2 – 5 εκρηκτικές επαναλήψεις. Οι 6 ώρες ανά μήνα προκύπτει με τη λογική ότι σε κάθε προπόνηση *plyometrics*, 20 – 30 λεπτά αντιστοιχούν σε αυτό το είδος προπόνησης και άλλα 45 – 60 λεπτά για ζέσταμα και αποθεραπεία [10], [11].

7) Άνω όριο για προπόνηση (σχετικά) υψηλής έντασης έτσι ώστε να μην υπάρχει υπερκόπωση-σχεδιαστική επιλογή (*HIIT, Resistance, Plyometrics*):

$$x_2 + x_3 + x_4 + x_8 \leq 28 \quad (3.9)$$

8) *Recovery ratio* για έντονη και πιο μέτριας έντασης προπόνηση:

$$x_2 + x_3 + x_4 + x_8 \leq 4(x_6 + x_7 + x_9 + 0.5x_5) \quad (3.10)$$

Δηλαδή, αριστερά προσθέτουμε τις ώρες έντονης προπόνησης και δεξιά τις ώρες ελαφριάς προπόνησης/ξεκούρασης. Να επισημάνουμε ότι πολλαπλασιάζουμε με 0.5 τη μεταβλητή x_5 και τη συμπεριλαμβάνουμε στη δεξιά πλευρά του ανισοτικού περιορισμού διότι πρώτον δεν πρόκειται για τόσο ελαφρύ τύπο ασκήσεων αλλά ούτε και τόσο βαρύ. Θα μπορούσε να ήταν και στην αριστερή πλευρά του εν λόγω περιορισμού.

9) Άνω όρια για προπόνηση άνω και κάτω μέρους, συμβατά με εκείνα που προτείνει η *ACSM (American Medicine & Science In Sports & Exercise)*:

$$x_3 \leq 18 \quad (3.11)$$

$$x_4 \leq 18 \quad (3.12)$$

- 10) Ισορροπία μεταξύ προπόνησης άνω και κάτω μέρους, προκειμένου να αποφύγουμε να εστιάσουμε τόσο πολύ στα πόδια ή μόνο στο στήθος/πλάτη/χέρια. Η τιμή 4 αποτελεί και σε αυτήν την περίπτωση σχεδιαστική επιλογή. Θα μπορούσε ο περιορισμός να ήταν και ισοτικός αλλά είναι προφανές ότι σε αυτήν την περίπτωση θα αυξανόταν η πιθανότητα να μην υπήρχε λύση στο πρόβλημά μας:

$$|x_3 - x_4| \leq 4 \quad (3.13)$$

- 11) Άνω όριο για προπόνηση *yoga*/ευλυγισίας:

$$x_7 \leq 12 \quad (3.14)$$

Αποτελεί περιορισμό πρακτικής φύσεως και όχι βασισμένος στη φυσιολογία του ανθρώπου και ουσιαστικά μας λέει για τον περιορισμό διαθεσιμότητας τέτοιων μαθημάτων. Παρόμοιου τύπου περιορισμοί θα μπορούσαν να επιβληθούν και στους άλλους τύπους ασκήσεων.

- 12) Περιορισμός ο οποίος μας εξασφαλίζει ότι δεν θα καταπονήσουμε πολύ τον κορμό μας (παρόμοιοι περιορισμοί θα μπορούσαν να διατυπωθούν και για τους άλλους τύπους προπονήσεων)[12]:

$$x_5 + x_9 \geq 8 \quad (3.15)$$

- 13) Περιορισμός ο οποίος είναι κρίσιμης σημασίας προκειμένου να μην αγνοήσουμε την προπόνηση *cardio* (μεταβλητή x_1) σε σχέση με την συνολική προπόνηση δύναμης (μεταβλητές x_3 και x_4). Επί της ουσίας, ο περιορισμός αυτός προκύπτει από το γεγονός ότι χρειαζόμαστε και αεροβική δύναμη αλλά και ενδυνάμωση, ενώ ο συντελεστής 0.8 τη στη δεξιά πλευρά μας λέει ότι δίνουμε λίγο παραπάνω έμφαση στη μυϊκή ενδυνάμωση:

$$x_1 \geq 0.8(x_3 + x_4) \quad (3.16)$$

14) Περιορισμοί μη αρνητικότητας των μεταβλητών του προβλήματος (αφού εκφράζουν ώρες ανά μήνα):

$$x_i \geq 0, i = 1, 2, \dots, 9 \quad (3.17)$$

15) Ελάχιστες ώρες για *active recovery* (ο περιορισμός αυτός εισήχθη αφού το πρόβλημα είχε λυθεί με όλους τους προηγούμενους περιορισμούς και διαπιστώθηκε ότι $x_9 = 0$) (3.18)

$$x_9 \geq 8 \quad (3.19)$$

Μέσω της μεθόδου *Simplex* (βλ. αρχείο *Lin_Com_Opt_Final_Project_v2.py*) βρίσκουμε ότι η λύση του εν λόγω προβλήματος είναι η εξής:

$$\{x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9\} = \{18, 8, 8, 12, 4, 0, 6, 0, 4\} \quad (3.20)$$

Δεν κάνουμε ανάλυση ευαισθησίας διότι έχουμε εκφυλισμένη λύση (όχι ότι κάτι τέτοιο δεν είναι εφικτό)

4. Δυϊκό πρόβλημα

Έχουμε συνολικά 16 περιορισμούς όποτε και 16 μεταβλητές απόφασης ($y_i, i = 1, 2, \dots, 16$).

Η αντικειμενική συνάρτηση που καλούμαστε να βελτιστοποιήσουμε προκύπτει κατά τα γνωστά:

$$\min(60y_1 + 10y_2 + 8y_3 + 6y_4 + 8y_5 + 6y_6 + 28y_7 + 18y_9 + 18y_{10} + 4y_{11} + 4y_{12} + 12y_{13} + 8y_{14} + 4y_{16} \quad (5.1)$$

Επίσης, για κάθε μεταβλητή απόφασης του πρωτεύοντος προβλήματος οι αντίστοιχοι περιορισμοί του δευτερεύοντος προβλήματος έχουν ως εξής:

$$x_1: y_1 + y_2 + y_{15} \geq 5 \quad (5.2)$$

$$x_2: y_1 + 2y_2 + 0.5y_3 + y_5 + y_7 + y_8 \geq 8 \quad (5.3)$$

$$x_3: y_1 + y_3 + y_7 + y_8 + y_9 + y_{11} - y_{12} - 0.8y_{15} \geq 6.5 \quad (5.4)$$

$$x_4: y_1 + y_3 + y_7 + y_8 + y_{10} - y_{11} + y_{12} - 0.8y_{15} \geq 6.5 \quad (5.5)$$

$$x_5: y_1 + 0.5y_3 - 2y_8 + y_{14} \geq 4 \quad (5.6)$$

$$x_6: y_1 + y_4 - 4y_8 \geq 2 \quad (5.7)$$

$$x_7: y_1 + 0.5y_2 + y_4 - 4y_8 + y_{13} \geq 3 \quad (5.8)$$

$$x_8: y_1 + 1.5y_2 + 0.5y_3 + y_6 + y_7 + y_8 \geq 5.5 \quad (5.9)$$

$$x_9: y_1 + 0.5y_2 - 4y_8 + y_{14} + y_{16} \geq 1.5 \quad (5.10)$$

Τέλος, για τους περιορισμούς προσήμου για κάθε μεταβλητή απόφασης y_i , θα ισχύει (ανάλογα κάθε φορά με το αν ο αντίστοιχος ανισοτικός περιορισμός του πρωτεύοντος είναι $\leq$ ή $\geq$):

$$y_1 \geq 0 \text{ (5.11)}$$

$$y_2 \leq 0 \text{ (5.12)}$$

$$y_3 \leq 0 \text{ (5.13)}$$

$$y_4 \leq 0 \text{ (5.14)}$$

$$y_5 \geq 0 \text{ (5.15)}$$

$$y_6 \geq 0 \text{ (5.16)}$$

$$y_7 \geq 0 \text{ (5.17)}$$

$$y_8 \geq 0 \text{ (5.18)}$$

$$y_9 \geq 0 \text{ (5.19)}$$

$$y_{10} \geq 0 \text{ (5.20)}$$

$$y_{11} \geq 0 \text{ (5.21)}$$

$$y_{12} \geq 0 \text{ (5.22)}$$

$$y_{13} \geq 0 \text{ (5.23)}$$

$$y_{14} \leq 0 \text{ (5.24)}$$

$$y_{15} \leq 0 \text{ (5.25)}$$

$$y_{16} \leq 0 \text{ (5.26)}$$

Και η λύση είναι (*βλ. αρχείο Lin_Com_Opt_Final_Project_v2.py*)

5. Σχολιασμός λύσεων πρωτεύοντος και δυϊκού προβλήματος

Figure 1 illustrates the sequence of pivots in the simplex algorithm, showing 14 nodes (0-13) arranged in a grid. Each node represents a phase I of the algorithm, with basic indices B , non-basic indices N , and a vector z .

The nodes are arranged in a grid, with nodes 0-3 in the top row, 4-7 in the second row, 8-11 in the third row, and 12-13 in the bottom row. The nodes are connected by arrows, indicating the sequence of pivots.

The nodes are labeled as follows:

- Node 0 (Phase I): $B = \{10, 12, 14, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 27, 28, 29, 31\}$, $N = \{s1, a2, a3, a4, s5, s6, s7, s8, s9, s10, s11, s12, s13, a14, a15, a16, a17, a18, a19, a20, a21, a22, a23, a24, a25, a26, a27, a28, a29, a30, a31, a32, a33, a34, s35, s36, s37, s38, s39, s40, s41, s42, s43, s44, s45, s46, s47, s48, s49, s50, s51, s52, s53, s54, s55, s56, s57, s58, s59, s60, s61, s62, s63, s64, s65, s66, s67, s68, s69, s70, s71, s72, s73, s74, s75, s76, s77, s78, s79, s80, s81, s82, s83, s84, s85, s86, s87, s88, s89, s90, s91, s92, s93, s94, s95, s96, s97, s98, s99, s100, s101, s102, s103, s104, s105, s106, s107, s108, s109, s110, s111, s112, s113, s114, s115, s116, s117, s118, s119, s120, s121, s122, s123, s124, s125, s126, s127, s128, s129, s130, s131, s132, s133, s134, s135, s136, s137, s138, s139, s140, s141, s142, s143, s144, s145, s146, s147, s148, s149, s150, s151, s152, s153, s154, s155, s156, s157, s158, s159, s160, s161, s162, s163, s164, s165, s166, s167, s168, s169, s170, s171, s172, s173, s174, s175, s176, s177, s178, s179, s180, s181, s182, s183, s184, s185, s186, s187, s188, s189, s190, s191, s192, s193, s194, s195, s196, s197, s198, s199, s200, s201, s202, s203, s204, s205, s206, s207, s208, s209, s210, s211, s212, s213, s214, s215, s216, s217, s218, s219, s220, s221, s222, s223, s224, s225, s226, s227, s228, s229, s230, s231, s232, s233, s234, s235, s236, s237, s238, s239, s240, s241, s242, s243, s244, s245, s246, s247, s248, s249, s250, s251, s252, s253, s254, s255, s256, s257, s258, s259, s260, s261, s262, s263, s264, s265, s266, s267, s268, s269, s270, s271, s272, s273, s274, s275, s276, s277, s278, s279, s280, s281, s282, s283, s284, s285, s286, s287, s288, s289, s290, s291, s292, s293, s294, s295, s296, s297, s298, s299, s300, s301, s302, s303, s304, s305, s306, s307, s308, s309, s310, s311, s312, s313, s314, s315, s316, s317, s318, s319, s320, s321, s322, s323, s324, s325, s326, s327, s328, s329, s330, s331, s332, s333, s334, s335, s336, s337, s338, s339, s340, s341, s342, s343, s344, s345, s346, s347, s348, s349, s350, s351, s352, s353, s354, s355, s356, s357, s358, s359, s360, s361, s362, s363, s364, s365, s366, s367, s368, s369, s370, s371, s372, s373, s374, s375, s376, s377, s378, s379, s380, s381, s382, s383, s384, s385, s386, s387, s388, s389, s390, s391, s392, s393, s394, s395, s396, s397, s398, s399, s400, s401, s402, s403, s404, s405, s406, s407, s408, s409, s410, s411, s412, s413, s414, s415, s416, s417, s418, s419, s420, s421, s422, s423, s424, s425, s426, s427, s428, s429, s430, s431, s432, s433, s434, s435, s436, s437, s438, s439, s440, s441, s442, s443, s444, s445, s446, s447, s448, s449, s450, s451, s452, s453, s454, s455, s456, s457, s458, s459, s460, s461, s462, s463, s464, s465, s466, s467, s468, s469, s470, s471, s472, s473, s474, s475, s476, s477, s478, s479, s480, s481, s482, s483, s484, s485, s486, s487, s488, s489, s490, s491, s492, s493, s494, s495, s496, s497, s498, s499, s500, s501, s502, s503, s504, s505, s506, s507, s508, s509, s510, s511, s512, s513, s514, s515, s516, s517, s518, s519, s520, s521, s522, s523, s524, s525, s526, s527, s528, s529, s530, s531, s532, s533, s534, s535, s536, s537, s538, s539, s540, s541, s542, s543, s544, s545, s546, s547, s548, s549, s550, s551, s552, s553, s554, s555, s556, s557, s558, s559, s560, s561, s562, s563, s564, s565, s566, s567, s568, s569, s570, s571, s572, s573, s574, s575, s576, s577, s578, s579, s580, s581, s582, s583, s584, s585, s586, s587, s588, s589, s590, s591, s592, s593, s594, s595, s596, s597, s598, s599, s600, s601, s602, s603, s604, s605, s606, s607, s608, s609, s610, s611, s612, s613, s614, s615, s616, s617, s618, s619, s620, s621, s622, s623, s624, s625, s626, s627, s628, s629, s630, s631, s632, s633, s634, s635, s636, s637, s638, s639, s640, s641, s642, s643, s644, s645, s646, s647, s648, s649, s650, s651, s652, s653, s654, s655, s656, s657, s658, s659, s660, s661, s662, s663, s664, s665, s666, s667, s668, s669, s670, s671, s672, s673, s674, s675, s676, s677, s678, s679, s680, s681, s682, s683, s684, s685, s686, s687, s688, s689, s690, s691, s692, s693, s694, s695, s696, s697, s698, s699, s700, s701, s702, s703, s704, s705, s706, s707, s708, s709, s710, s711, s712, s713, s714, s715, s716, s717, s718, s719, s720, s721, s722, s723, s724, s725, s726, s727, s728, s729, s730, s731, s732, s733, s734, s735, s736, s737, s738, s739, s740, s741, s742, s743, s744, s745, s746, s747, s748, s749, s750, s751, s752, s753, s7$

Παρατηρώντας το παραπάνω, βλέπουμε ότι απαιτείται ένα σχετικά μεγάλο (16) πλήθος επαναλήψεων προκειμένου να λυθεί το αρχικό πρόβλημα κάτι το οποίο οφείλεται προφανώς στην αυξημένη πολύπλοκότητά του όπως επίσης και στο γεγονός ότι χρειάστηκε να αξιοποιήσουμε την μέθοδο *Simplex* δύο φάσεων, ώστε να είναι δυνατή η εκκίνηση της *Simplex*, μιας και δε μπορούμε να βρούμε άμεσα προφανή εφικτή λύση. Η δε μοναδική διαδρομή που έχει σχεδιαστεί στον **SAG**, δικαιολογείται από το γεγονός ότι αντι να χρησιμοποιήσουμε

έναν τυχαίο κανόνα για την επιλογή της εισερχόμενης μεταβλητής (με θετικό όμως αντικειμενικό συντελεστή), επιλέγουμε εκείνη με το μεγαλύτερο *reduced cost* [14]. Επίσης, ενδιαφέρον παρουσιάζουν και οι τιμές των μεταβλητών απόφασης για κάθε πρόβλημα. Ειδικότερα, στο πρωτεύον πρόβλημα, οι μεταβλητές $x_5 = 0$ και $x_8 = 0$, πράγμα το οποίο σημαίνει ότι μπορούμε να πραγματοποιήσουμε τον στόχο μας (δηλαδή τη μεγιστοποίηση του *fitness score*), με το να μη κάνουμε ορισμένους τύπους προπονήσεων (*core* και *plyometrics* στη συγκεκριμένη περίπτωση). Το τελευταίο δεν προκαλεί ιδιαίτερη έκπληξη μιας και οι διάφοροι τύποι ασκήσεων να μην εστιάζουν σε ένα συγκεκριμένο κομμάτι (πχ προπόνηση δύναμης στη δύναμη) αλλά επηρεάζουν και άλλα κομμάτια (σε μικρότερο βαθμό προφανώς), έτσι ώστε κάποιοι τύποι προπονήσεων να μην είναι απαραίτητοι, λόγω των συσσωρευτικών μικρών επιδράσεων που έχουν ορισμένες ασκήσεις. Βέβαια, το όλο σκηνικό έχει να κάνει και με το μοντέλο που έχουμε στήσει (αποτελεί μια υπεραπλούστευση μιας και δε λαμβάνει υπόψιν του πολλά είδη ασκήσεων, εξωτερικούς παράγοντες, την φυσιολογία του κάθε ανθρώπου ξεχωριστά κλπ) και είναι πιθανόν (σίγουρο) διαφορετικές εκδοχές του (θα εξεταστούν παρακάτω) έχουν διαφορετική λύση.

Τώρα, ως πιο ειδικά σχόλια, είναι προφανές ότι κάθε μεταβλητή απόφασης στο δυϊκό πρόβλημα μας λέει εάν αυξήσουμε το δεξί μέρος του αντίστοιχου περιορισμού κατά μία μονάδα, μας λέει την ποσότητα κατά την οποία θα αλλάξει η τιμή του *fitness score* (αντικειμενικής συνάρτησης- δεν χρειάζεται το πρόβλημα να βρίσκεται σε τυπική μορφή). Μηδενικές τιμές στις μεταβλητές του δυϊκού προβλήματος μας λένε ότι η βέλτιστη τιμή της αντικειμενικής μας συνάρτησης δεν πρόκειται να αλλάξει αν μεταβάλλουμε το δεξί μέλος του εκάστοτε περιορισμού κατά μία μονάδα. Επίσης, εάν η εν λόγω μεταβλητή είναι αρνητική τότε έχουμε μείωση στην αντικειμενική συνάρτηση ενώ εάν είναι θετική, αύξηση.

6. Επίλυση ενός πιο περίπλοκου προβλήματος

Έχοντας καταστρώσει και επιλύσει το πρόβλημά του *fitness scheduling*, εδώ θα ασχοληθούμε με ένα παρόμοιας λογικής πρόβλημα το οποίο περιγράφεται στο [15]. Ειδικότερα, στόχος μας τώρα είναι πάλι η διαμόρφωση ενός προγράμματος γυμναστικής, αποφασίζοντας ποιες ασκήσεις θα συμπεριλάβουμε και ποιες όχι όπως επίσης και την σειρά των ασκήσεων έτσι ώστε ο συνολικός χρόνος προπόνησης να είναι ο ελάχιστος ενώ παράλληλα ικανοποιούνται ορισμένες απαιτήσεις έντασης για τον κάθε τύπο μυ. Επιλέγουμε N ασκήσεις από ένα *candidate* σετ ασκήσεων E .

a) Ορισμός μεταβλητών απόφασης:

Για κάθε τύπο άσκηση $i \in E$ και κάθε ζεύγος ασκήσεων $i, j \in E, i \neq j$:

$x_i \in \{0,1\}$: 1 εάν η άσκηση i περιλαμβάνεται στο πρόγραμμα, 0 ειδάλλως (7.1)

$f_{ij} \in \{0,1\}$: 1 εάν η άσκηση j ακολουθεί αμέσως μετά την άσκηση i , 0 ειδάλλως (7.2)

$u_i \in N$, δείκτης για τη σειρά της άσκησης (0 εάν υπάρχει στο πρόγραμμα)

b) Δεδομένα/παράμετροι του προβλήματος:

d_i : διάρκεια της άσκησης i

t_{ij} : χρόνος μετάβασης από την άσκηση i στην άσκηση j (ξεκούραση, προετοιμασία)

N : αριθμός ασκήσεων που θα συμπεριληφθούν στο πρόγραμμα

M : σετ των μυϊκών ομάδων

w_{im} : μας λέει εάν για τη μυϊκή ομάδα m , η άσκηση i είναι κύρια ($w_{im} = 2$),

δευτερεύουσα ($w_{im} = 1$) ή σε διαφορετική περίπτωση

v_{im} : μας λέει εάν η μυϊκή ομάδα m χρησιμοποιείται στην άσκηση i (είτε ως κύρια είτε ως δευτ.)
(υπολογίζεται από το w_{im})

$MET(i)$: ένταση (σε METS) της άσκησης i

T_{MET} : παράμετρος η οποία χρησιμοποιείται προκειμένου μία άσκηση να ταξινομηθεί ως χαμηλής ή ως υψηλής έντασης

c) Αντικειμενική συνάρτηση:

$$\min \sum_{i \in E} d_i x_i + \sum_{\substack{i, j \in E, \\ i \neq j}} t_{ij} f_{ij} \quad (7.3)$$

d) Περιορισμοί προβλήματος

1) Επιλογή N ασκήσεων συνολικά:

$$\sum_{i \in E} x_i = N \quad (7.4)$$

2) Δημιουργία $N - 1$ τόξων στο μονοπάτι των ασκήσεων (άσκηση $a \rightarrow b \rightarrow c$ κλπ:

$$\sum_{\substack{i, j \in E, \\ i \neq j}} f_{ij} = N - 1 \quad (7.5)$$

3) Το τόξο $i \rightarrow j$ επιτρέπεται μόνο όταν και τα δύο άκρα του υπάρχουν στο πρόγραμμα:

$$f_{ij} \leq x_i \quad (7.6)$$

$$f_{ij} \leq x_j \quad (7.7)$$

4) Περιορισμοί για τις το μονοπάτι των ασκήσεων που θα ακολουθήσουμε το οποίο μας λέει ότι υπάρχει το πολύ ένας προηγούμενος και ένας επόμενος κόμβος (άσκηση):

$$\sum_{\substack{j \in E. \\ j \neq i}} f_{ij} \leq 1 \quad (7.8)$$

$$\sum_{\substack{j \in E. \\ j \neq i}} f_{ji} \leq 1 \quad (7.9)$$

- 5) Περιορισμός ο οποίος μας εξασφαλίζει ότι δεν θα ξεκινήσουμε από μία άσκηση και θα καταλήξουμε στην ίδια άσκηση:

$$s_i - s_j + (N - 1)f_{ij} \leq N, \forall i, j \in E, i \neq j \quad (7.10)$$

- 6) Περιορισμός για την σειρά των ασκήσεων

$$0 \leq s_i \leq N, \forall i \in E \quad (7.11)$$

- 7) Περιορισμός ο οποίος κάνει *force* τη μεταβλητή $s_i = 0$ (7.12), εάν η άσκηση i , δεν συμπεριλαμβάνεται στο πρόγραμμα:

$$s_i \leq N \sum_{j \neq i} (f_{ij} + f_{ji}), \forall i \in E \quad (7.13)$$

Η λογική του παραπάνω περιορισμού είναι ότι εάν η άσκηση i περιλαμβάνεται στο πρόγραμμα η *RHS* της παραπάνω ανίσωσης είναι μηδέν, δηλαδή $s_i \leq 0$, οπότε $s_i = 0$ (7.14). Σε διαφορετική περίπτωση η *RHS* είναι $\geq N$, οπότε η μεταβλητή s_i είναι ελεύθερη

- 8) Περιορισμός ο οποίος μας εξασφαλίζει (σε συνδυασμό με τους προηγούμενους) ότι το *indexing* που χρησιμοποιείται για τις ασκήσεις στο πρόγραμμα είναι $\{1, 2, \dots, N\}$ χωρίς κενά, δηλαδή να μην επιτρέπεται πχ $1 \rightarrow 2 \rightarrow 4$:

$$\sum_{i \in E} s_i = \frac{N(N + 1)}{2} \quad (7.14)$$

- 9) Περιορισμός έτσι ώστε να μην καταπονούμε πολύ την ίδια μυϊκή ομάδα σε δύο διαδοχικές ασκήσεις:

$$f_{ij} = 0 - \text{για } w_{im} = w_{jm} = 2 \quad (7.15)$$

- 10) Περιορισμός ο οποίος κάνει force μία μυϊκή ομάδα να χρησιμοποιηθεί μετά από δύο διαδοχικές χρήσεις:

για οποιαδήποτε μυϊκή ομάδα m , και για οποιεσδήποτε 3 ασκήσεις i, j, k με $v_{im} = v_{jm} = v_{km} = 1$):

$$f_{ij} + f_{jk} \leq 1 \quad (7.16)$$

Η δε γενίκευση του παραπάνω περιορισμού έχει ως εξής:

Για οποιεσδήποτε p ασκήσεις $i_1, i_2, \dots, i_p$ οι οποίες χρησιμοποιούν όλες τους τη μυϊκή ομάδα m :

$$f_{i_1 i_2} + f_{i_2 i_3} + \dots + f_{i_{p-1} i_p} \leq p - 2 \quad (7.17)$$

- 11) Περιορισμός τον οποίο θέτουμε έτσι ώστε σε μία προπόνηση, ούτε να μην εξασκούμε καθόλου αλλά και ούτε να υπερκαταπονούμε μία συγκεκριμένη μυϊκή ομάδα:

$$LB_{muscle} \leq \sum_{i \in E} w_{im} x_i \leq UB_{muscle} \quad \text{για κάθε μυϊκή ομάδα } m \quad (7.18)$$

- 12) Περιορισμός για το μέσο MET όλων των ασκήσεων (δεν περιλαμβάνεται στον κώδικα):

$$LB_{avgMET} \leq \frac{1}{N} \sum_{i \in E} MET_i \leq UB_{avgMET} \quad (7.19)$$

- 13) Περιορισμός ο οποίος απαγορεύει διαδοχικές ασκήσεις υψηλής έντασης:

$$H = \{i \in E : MET_i \geq T_{MET}\}$$

$$\forall i, j \in H, i \neq j : f_{ij} = 0 \quad (7.20)$$

Μερικές παράμετροι του προβλήματος που χρησιμοποιήθηκαν

E = bench_press, chest_fly, dips, push_up, pull_up, shoulder_press, upright_row, bent_over_row, lat_pulldown, biceps_curl, deadlift, squat, back_extension, crunch, exercise_wheel, side_plank, leg_press, lunge, push_up_rotation, jumping_jack, jumping_squat, burpee, mountain_climber, plyo_lunge, lunge_jump, step_up, battle_rope

M = chest, shoulders, triceps, biceps, lats, trapezius, lower_back, abs, quads, glutes_hams

$t_{ij} = 0.12$ ($i \neq j$) με τις εξής εξαιρέσεις:

(exercise_wheel, jumping_squat)=0.25

(jumping_squat, exercise_wheel)=0.25

(upright_row, biceps_curl)=0.20

(deadlift, squat)=0.20

(squat, deadlift)=0.20

(bench_press, chest_fly)=0.18

(chest_fly, bench_press)=0.18

(pull_up, lat_pulldown)=0.22

(lat_pulldown, pull_up)=0.22

(battle_rope, biceps_curl)=0.20

(bent_over_row, deadlift)=0.22

(deadlift, bent_over_row)=0.22

Εν τέλει η λύση στην οποία καταλήγουμε έχει ως εξής (βλ. αρχείο ***Lin_Com_Opt_Final_Project_adv_v1.py***):

```
x[bench_press] = 1
x[chest_fly] = 1
x[dips] = 1
x[push_up] = 1
x[pull_up] = 1
x[shoulder_press] = 1
x[upright_row] = 1
x[bent_over_row] = 1
x[lat_pulldown] = 1
x[biceps_curl] = 1
x[deadlift] = 1
x[squat] = 1
x[back_extension] = 1
x[crunch] = 1
x[exercise_wheel] = 1
x[side_plank] = 1
x[leg_press] = 1
x[lunge] = 1
x[push_up_rotation] = 1
x[jumping_jack] = 1
x[jumping_squat] = 1
x[burpee] = 1
x[mountain_climber] = 1
x[plyo_lunge] = 1
x[lunge_jump] = 1
x[step_up] = 1
x[battle_rope] = 1

irc count chosen: 26 (expected N-1 = 26 )

'ath (ordered):
plyo_lunge -> chest_fly -> squat -> push_up_rotation -> lunge_jump -> upright_row -> lunge -> bench_press -> bent_over_
ow -> burpee -> shoulder_press -> exercise_wheel -> dips -> crunch -> back_extension -> mountain_climber -> lat_pulldown
-> deadlift -> push_up -> jumping_squat -> biceps_curl -> leg_press -> battle_rope -> side_plank -> jumping_jack -> step
up -> pull_up
```

7. Σύγκριση/σχολιασμός των δύο τύπων προβλημάτων (απλού και περίπλοκου)

Αναφορικά με το πρώτο πρόβλημα (μεγιστοποίηση *fitness score*) τρέχοντας τον σχετικό κώδικα παρατηρούμε ότι απαιτούνται περίπου 6s και 125.26MBytes μνήμης, ενώ για το 2^ο πρόβλημα που είναι τάξεις μεγέθους πιο περίπλοκο, απαιτούνται περίπου 1s και 74.9 MBytes μνήμης (τάξη μεγέθους). Βέβαια, αυτό προκύπτει από το ότι στην πρώτη περίπτωση αναλύουμε το πρόβλημα διεξοδικότερα αφού (στον 1^ο κώδικα-**αρχείο** **Lin_Com_Opt_Final_Project_normal_v2.py**):

- a) Εκτυπώνουμε σε κάθε επανάληψη το *Simplex tableau*
- b) Εκτυπώνουμε το *SAG*
- c) Λύνουμε το δυϊκό πρόβλημα

, ενώ στο 2^ο – πιο *advanced* πρόβλημα το μόνο που κάνουμε να λύσουμε το πρόβλημα. Αν το επιχειρούσαμε το παραπάνω, ο χρόνος εκτέλεσης και οι απαιτούμενοι πόροι θα ήταν αυξημένοι κατά πολύ. Ας αναλύσουμε όμως το γιατί:

Αρχικά έχουμε 27 μεταβλητές x_i αφού το σύνολο E έχει 27 στοιχεία. Μετά έχουμε άλλες $27 \times 26 = 702$ μεταβλητές f_{ij} (που δείχνουν αν η άσκηση j ακολουθεί την i). Τέλος έχουμε άλλες 27 μεταβλητές s_i οι οποίες μα δείχνουν την σειρά των ασκήσεων, επομένως συνολικά έχουμε $27 + 702 + 27 = 756$ μεταβλητές (εξαιρουμένων των παραμέτρων – βλ. σελ. 15–) οι οποίες ρυθμίζονται χειροκίνητα

Ας δούμε όμως και τους περιορισμούς:

- 1) Επιλογή $N = 27$ ασκήσεων-1 περιορισμός

- 2) Επιλογή $N - 1 = 26$ τόξων ασκήσεων- 1 περιορισμός
- 3) Περιορισμοί (7.6), (7.7) – $2 \times 27 \times 27 = 1404$ περιορισμοί
- 4) Περιορισμός (7.8) – 27 περιορισμοί
- 5) (7.9) – 27
- 6) (7.10) – 702 (1 για κάθε διατεταγμένο ζεύγος)
- 7) (7.11) – 27
- 8) (7.14) – 1 περιορισμός
- 9) (7.15) – 144 περιορισμοί (το νούμερο αυτό προκύπτει με τη λογική ότι για μυϊκή ομάδα m, μετράμε πόσες ασκήσεις χρησιμοποιούν την εν λόγω μυϊκή ομάδα με $w_{im} = 2$ (8.2), και το πολλαπλασιάζουμε με τον εαυτό του μειωμένου κατά 1, προκειμένου να βρούμε το συνολικό αριθμό απαγορευμένων ζευγών ασκήσεων
- 10) (7.16) – 3264 περιορισμοί – πάλι με την ίδια λογική, μόνο που εδώ μετράμε των αριθμό των ασκήσεων οι οποίες χρησιμοποιούν τη μυϊκή ομάδα m (έστω a) είτε ως κύρια είτε ως δευτερεύουσα) και εν συνεχεία βρίσκουμε το γινόμενο $a * (a - 1) * (a - 2)$
- 11) (7.18) – $2 * 10 = 20$ περιορισμοί (αφού έχουμε 10 μυϊκές ομάδες)
- 12) (7.19) – 2 περιορισμοί (*upper και lower bound*)
- 13) (7.20) – $10 * 9 = 90$ περιορισμοί

Επομένως στο σύνολο είναι $1 + 1 + 1404 + 27 + 27 + 702 + 27 + 1 + 144 + 3264 + 20 + 2 + 90 = 5710$ περιορισμοί

Σε κάθε περίπτωση η μέθοδος *Simplex* σε προβλήματα της πραγματικής ζωής [14] (όπως το παρόν) θέλει πολυωνυμικό χρόνο (εξαρτώμενο από τον αριθμό των μεταβλητών απόφασης και των περιορισμών), αλλά γίνεται κατανοητό ότι εάν εφαρμόζαμε ένα από τα a), b), c) τα πράγματα από άποψη χρόνου θα ξέφευγαν, ειδικά άμα οι διαθέσιμοι υπολογιστικοί πόροι είναι περιορισμένοι.

8. Πιθανές τροποποιήσεις/διαφορετικές τεχνικές επίλυσης στα δύο προβλήματα

Και στα δύο προβλήματα αντί για μία συνάρτηση που θέλουμε να βελτιστοποιήσουμε την τιμή της, θα μπορούσαμε να είχαμε πολλές επιμέρους (*fitness score*, συνολικός χρόνος, πιθανότητα τραυματισμού, στόχος για καρδιακούς παλμούς ανά άσκηση κλπ) αλλά αυτό προϋποθέτει ότι εκτοξεύεται ο αριθμός των μεταβλητών απόφασης και των ισοτικών/ανισοτικών περιορισμών, με αποτέλεσμα το υπολογιστικό κόστος να εκτοξεύεται. Στους περιορισμούς επίσης θα μπορούσαν να ενταχθούν (περισσότερο) περιορισμοί που αφορούν στα *logistics*/διαθεσιμότητα εξοπλισμού/*instructors* κλπ αλλά και την αβεβαιότητα η οποία υπεισέρχεται σε διάφορους τομείς του προβλήματος (πχ *MET* για κάθε άσκηση, εύρος κίνησης κλπ). Σε μια ακόμα πιο ρεαλιστική εκδοχή, θα ήταν δυνατόν το όλο πρόγραμμα ασκήσεων να το προσαρμόσουμε στο εκάστοτε άτομο, έχοντας ενσωματωμένες παραμέτρους οι οποίες μας λένε και πόση άσκηση επιπλέον χρειάζεται/σε ποια μυϊκή ομάδα πρέπει να δώσουμε πιο πολύ βάρος αλλά και την φυσιολογία/τρόπο ζωής του ατόμου.

Τέλος παραθέτουμε (μόνο ονομαστικά μιας και η σχετική ανάλυση ξεφεύγει από τους σκοπούς της παρούσης εργασίας) διαφορετικούς αλγορίθμους επίλυσης των παραπάνω δύο προβλημάτων [15]-[18]) :

-Interior-point (barrier / primal–dual IPM)

-Branch-and-bound / branch-and-cut

-Assignment / Hungarian (position models, relaxations), **min-cost flow** (if reducible)

-SPPRC (shortest path with resource constraints; label-setting/label-correcting)

-Dynamic programming (knapsack/time-indexed substructures)

-Benders decomposition (structure: selection vs. sequencing/scheduling)

-Dantzig–Wolfe / column generation; branch-and-price on patterns/routes

- Lagrangian relaxation + subgradient** (relax hard spacing/quotas; dualize)
- ADMM** (decompose by muscles/modalities vs. calendar/sequence)
- Constraint Programming (CP-SAT)** for precedences, spacing, no-overlap, calendars
- Heuristics / Metaheuristics:** greedy + local search; 2-opt/3-opt/Lin-Kernighan;
ALNS/LNS, Tabu, Simulated Annealing, GRASP, VNS/VND, GA/NSGA-II, PSO
- Matheuristics / Hybrids:** CP+MIP, fix-and-optimize, local branching, RINS, rolling horizon
- **Stochastic/Robust optimization:** SAA (scenario sampling), chance constraints, robust counterparts
- Learning-based:** reinforcement learning / bandits (adaptive plans with hard-constraint guards)

9. Συμπεράσματα-Γενικός σχολιασμός

Ως ένα γενικό συμπέρασμα μπορούμε να πούμε ότι με βάση τα παραπάνω, η μέθοδος Simplex πρόκειται για το πλέον ισχυρό εργαλείο επίλυσης προβλημάτων γραμμικού προγραμματισμού, έχει όμως αυξημένες απαιτήσεις σε μνήμη (κάτι το οποίο γίνεται άμεσα αντιληπτό από το πρώτο και πιο απλό πρόβλημα όπου απαιτήθηκαν δεκάδες *MBytes*). Εντούτοις, δε πρέπει να λησμονούμε το γεγονός ότι εξαιτίας ακριβώς των περιορισμών, της αντικειμενικής συνάρτησης και των μεταβλητών απόφασης, να μην υπάρχει λύση στο εκάστοτε πρόβλημα, κάτι το οποίο προϋποθέτει την κατάλληλη επιλογή/διαμλορφωση των τελευταίων όπως επίσης και των παραμέτρων που εμπλέκονται στο πρόβλημα. Τέλος, και τα δύο μοντέλα είναι «μοντέλα» υπό την έννοια ότι παρεκκλίνουν (αρκετά από την πραγματικότητα) κάτι το οποίο οφείλεται στο γεγονός ότι το οποιοδήποτε πρόβλημα να μοντελοποιείται ελλειπώς αναφορικά με τις μεταβλητές απόφασης, την αντικειμενική συνάρτηση, τους περιορισμούς, της διάφορες προσεγγίσεις που γίνονται, την επιλογή των παραμέτρων αλλά και –ειδικά για το παρόν- την πιθανή ανακρίβεια των διάφορων προταθέντων κανόνων αλλά και τη λειτουργία του ανθρώπου ως το πλέον μη-γραμμικό σύστημα.

10 Κώδικας

Απαιτούμενες βιβλιοθήκες που πρέπει να εγκατασταθούν (μέσω *cmd* –Windows):

numpy, pandas, matplotlib, pulp, psutil

Σύντομη ανάλυση του κώδικα (υπάρχουν και πιο αναλυτικά comments στον κώδικα *per se* για το 1^ο πρόβλημα (αρχείο *Lin_Com_Opt_Final_Project_normal_v2.py*):

1. Εγκατάσταση απαραίτητων βιβλιοθηκών
2. Μοντελοποίηση δεδομένων προβλήματος
3. Επίλυση προβλήματος με *pulp* για επιβεβαίωση
4. Επίλυση προβλήματος με μία *custom* μέθοδο *Simplex*
5. Δημιουργία *SAG*
6. Ελέγχουμε ποιοι περιορισμοί είναι δεσμευτικοί και ποιοι όχι
7. Επίλυση δυαδικού προβλήματος
8. Εκτύπωση πόρων μνήμης και χρόνου

Για το 2^ο πρόβλημα:

Ουσιαστικά κάνουμε τα ίδια πράγματα με εξαίρεση τα 5,6, 7

Παραθέτουμε και τους κώδικες:

1^ο Πρόβλημα

```
# =====  
# WORKOUT LP — TWO-PHASE SIMPLEX + TABLEUX + READABLE SAG + SA (x1..x9 ONLY) + DUAL  
# =====  
  
# • Builds a 9-variable "hours per month" workout LP (x1..x9 ≥ 0).  
# • Solves with PuLP/CBC quickly (sanity check).  
# • Runs a FROM-SCRATCH TWO-PHASE SIMPLEX (tableau):  
#   PHASE I : add artificials, maximize -sum(artificials) to get a feasible basis.  
#   PHASE II : drop artificials, optimize the true objective.  
#   Every pivot prints ENTER/LEAVE and a full tableau with '-Z' (reduced costs).  
# • Saves and shows a readable Simplex Adjacency Graph (SAG) to the current folder.  
# • Extracts B, B-1, N, cB, cN, reduced costs and dual prices yT = cBT B-1.  
# • Performs Sensitivity Analysis using the EXACT slide formulas — but ONLY for the  
#   decision variables x1..x9 (as requested). We EXCLUDE slacks/surpluses/artificials  
#   from (A) and (B):  
#   (A) If xα is BASIC: (cNT - (cB + δ ek)T B-1 N)α ≤ 0  ∀ nonbasic decision cols α  
#       → interval for δ; ΔZ = δ · x{B,k}.  
#   (B) If xα is NON-BASIC: (only its reduced cost changes)  
#        $\bar{r}_\alpha(\delta) = \bar{r}_\alpha + \delta \leq 0 \Rightarrow \delta \leq -\bar{r}_\alpha$ , where α refers ONLY to that decision column.  
#   (C) RHS perturbation bi ← bi + γ:  
#       B-1 b + γ B-1 ei ≥ 0 (standard; independent of (A)/(B) restriction).  
# =====  
  
import math  
import os  
import numpy as np  
import pandas as pd  
import matplotlib  
import matplotlib.pyplot as plt  
  
# =====
```

```

# (PERF) Timing & Memory utilities
# =====

import sys, time, platform

try:
    import psutil # best effort for current RSS
except Exception:
    psutil = None

try:
    import resource # best effort for peak RSS (ru_maxrss)
except Exception:
    resource = None

_SCRIPT_TO = time.perf_counter()

def _bytes_to_mb(x):
    return f"{x / (1024*1024):.2f} MB" if isinstance(x, (int, float)) and x is not None else "N/A"

def get_current_rss_bytes():
    """Return current resident set size in bytes, if available."""
    # Prefer psutil if present
    if psutil is not None:
        try:
            return psutil.Process(os.getpid()).memory_info().rss
        except Exception:
            pass
    # Try /proc/self/statm on Linux
    try:
        if sys.platform.startswith("linux"):
            with open("/proc/self/statm", "r") as f:
                parts = f.read().split()
                if len(parts) >= 2:
                    pages = int(parts[1])
                    page_size = os.sysconf("SC_PAGE_SIZE")
                    return pages * page_size
        except Exception:
            pass
    return None # Unknown

```

```

def get_peak_rss_bytes():
    """Return peak RSS in bytes if available (based on ru_maxrss semantics)."""
    if resource is None:
        return None
    try:
        ru = resource.getrusage(resource.RUSAGE_SELF)
        peak = getattr(ru, "ru_maxrss", None)
        if peak is None:
            return None
        # On Linux, ru_maxrss is in kilobytes; on macOS it's in bytes.
        if sys.platform.startswith("darwin"):
            return int(peak) # bytes
        else:
            return int(peak * 1024) # KB → bytes
    except Exception:
        return None

def print_perf(tag, t0):
    """Print wall time and memory (current + peak RSS) for a section."""
    dt = time.perf_counter() - t0
    cur = get_current_rss_bytes()
    peak = get_peak_rss_bytes()
    print(f"[PERF] {tag}: {dt:.6f} s | RSS now={_bytes_to_mb(cur)} | Peak RSS={_bytes_to_mb(peak)}")

# -----
# (0) MODEL DATA
# -----
# Decision variables (monthly hours; all >= 0)
# x1: Moderate cardio
# x2: HIIT (vigorous intervals)
# x3: Strength upper
# x4: Strength lower
# x5: Core & stability
# x6: Flexibility / stretching
# x7: Yoga / mobility
# x8: Plyometrics / agility
# x9: Active recovery (very light movement)
var_names = ["x1", "x2", "x3", "x4", "x5", "x6", "x7", "x8", "x9"]

```

```

# Objective weights (points per hour). Tweak to taste.
c = np.array([5.0, 8.0, 6.5, 6.5, 4.0, 2.0, 3.0, 5.5, 1.5], dtype=float)

# Minimum explicit Active Recovery to avoid x9 = 0
MIN_ACTIVE_RECOVERY = 8.0 # e.g., ≈ 2 h/week baseline

# Build constraints A x (sense) b
A_list, b_list, sense_list = [], [], []

# C1) total time ≤ 60
A_list.append([1,1,1,1,1,1,1,1]); b_list.append(60.0); sense_list.append("<=")

# C2) aerobic min ≥ 10.8 (vigorous=2x; plyo≈1.5x; yoga/active=0.5x)
A_list.append([1,2,0,0,0,0,0.5,1.5,0.5]); b_list.append(10); sense_list.append(">=")

# C3) strength min ≥ 8 (core/HIIT/plyo half-credit toward "strengthiness")
A_list.append([0,0.5,1,1,0.5,0,0,0.5,0]); b_list.append(8.0); sense_list.append(">=")

# C4) flexibility + mobility ≥ 6
A_list.append([0,0,0,0,0,1,1,0,0]); b_list.append(6.0); sense_list.append(">=")

# C5) HIIT cap ≤ 8
A_list.append([0,1,0,0,0,0,0,0,0]); b_list.append(8.0); sense_list.append("<=")

# C6) plyometrics cap ≤ 6
A_list.append([0,0,0,0,0,0,0,1,0]); b_list.append(6.0); sense_list.append("<=")

# C7) "heavy" cap ≤ 28 (HIIT + both strength + plyo)
A_list.append([0,1,1,1,0,0,0,1,0]); b_list.append(28.0); sense_list.append("<=")

# C8) fatigue balance: (x2+x3+x4+x8) - 4(x6+x7+x9) - 2x5 ≤ 0
A_list.append([0,1,1,1,-2,-4,-4,1,-4]); b_list.append(0.0); sense_list.append("<=")

# C9) strength upper ≤ 18
A_list.append([0,0,1,0,0,0,0,0,0]); b_list.append(18.0); sense_list.append("<=")

# C10) strength lower ≤ 18
A_list.append([0,0,0,1,0,0,0,0,0]); b_list.append(18.0); sense_list.append("<=")

# C11) upper - lower ≤ 4
A_list.append([0,0,1,-1,0,0,0,0,0]); b_list.append(4.0); sense_list.append("<=")

# C12) lower - upper ≤ 4
A_list.append([0,0,-1,1,0,0,0,0,0]); b_list.append(4.0); sense_list.append("<=")

# C13) yoga supply cap ≤ 12
A_list.append([0,0,0,0,0,0,1,0,0]); b_list.append(12.0); sense_list.append("<=")

# C14) core + active ≥ 8
A_list.append([0,0,0,0,1,0,0,0,1]); b_list.append(8.0); sense_list.append(">=")

```

```

# C14b) explicit Active Recovery  $\geq$  MIN_ACTIVE_RECOVERY
A_list.append([0,0,0,0,0,0,0,1]); b_list.append(MIN_ACTIVE_RECOVERY); sense_list.append(">=")

# C15) cardio near strength:  $x_1 - 0.8x_3 - 0.8x_4 \geq 0$ 
A_list.append([1,0,-0.8,-0.8,0,0,0,0]); b_list.append(0.0); sense_list.append(">=")

A = np.array(A_list, dtype=float)
b = np.array(b_list, dtype=float)

m, n = A.shape # m = number of constraints (15), n = number of decision vars (9)

# -----
# (1) Quick PRIMAL solve with PuLP/CBC (sanity check)
# -----

t_cbc = time.perf_counter()

try:
    import pulp

    prob = pulp.LpProblem("WorkoutLP", pulp.LpMaximize)

    x = {nm: pulp.LpVariable(nm, lowBound=0) for nm in var_names}

    # Objective

    prob += pulp.lpSum(w*x[nm] for w, nm in zip(c, var_names))

    # Constraints

    for i in range(m):
        expr = pulp.lpSum(A[i,j]*x[var_names[j]] for j in range(n))

        if sense_list[i] == "<=": prob += expr <= b[i]
        elif sense_list[i] == ">=": prob += expr >= b[i]
        else:
            prob += expr == b[i]

    prob.solve(pulp.PULP_CBC_CMD(msg=False))

    print("CBC status:", pulp.LpStatus[prob.status])

    print("Primal Z* (CBC):", pulp.value(prob.objective))

    for nm in var_names: print(f" {nm} = {x[nm].value():.6g}")

    print_perf("CBC (primal sanity)", t_cbc)
except Exception as e:
    print("PuLP/CBC not available:", e)

    print_perf("CBC attempt (failed)", t_cbc)

# -----
# (2) Two-phase Simplex (tableau) — we want to see all steps
# -----

def make_standard_form(A, b, sense, var_names):

```

```
"""
```

Convert Ax (sense) b, $x \geq 0$ to equality form for a tableau:

- ensure RHS ≥ 0 (flip row if needed),
- add SLACK for $\leq$, SURPLUS+ARTIFICIAL for $\geq$, ARTIFICIAL for $=$,
- in Phase I, artificials are basic and cost is -1 to drive them to 0.

Returns:

A_std, b_std, all_col_names, basis0(list of basic col indices),
art_cols(indices of artificial columns), c_phase1 (cost for Phase I).

```
"""
```

```
m, n = A.shape
```

```
A_std = A.copy(); b_std = b.copy(); sense = sense.copy()
```

```
names = var_names.copy()
```

```
basis, art_cols = [], []
```

```
# Make RHS  $\geq 0$ 
```

```
for i in range(m):
```

```
    if b_std[i] < 0:
```

```
        A_std[i,:] *= -1; b_std[i] *= -1
```

```
        if sense[i] == "<=": sense[i] = ">="
```

```
        elif sense[i] == ">=": sense[i] = "<="
```

```
# Add columns row-by-row according to the sense
```

```
for i in range(m):
```

```
    if sense[i] == "<=":
```

```
        col = np.zeros((m,1)); col[i,0] = 1.0 # slack
```

```
        A_std = np.hstack((A_std, col)); names.append(f"s{i+1}")
```

```
        basis.append(A_std.shape[1]-1) # slack basic
```

```
    elif sense[i] == ">=":
```

```
        col = np.zeros((m,1)); col[i,0] = -1.0 # surplus
```

```
        A_std = np.hstack((A_std, col)); names.append(f"u{i+1}")
```

```
        col = np.zeros((m,1)); col[i,0] = 1.0 # artificial
```

```
        A_std = np.hstack((A_std, col)); names.append(f"a{i+1}")
```

```
        basis.append(A_std.shape[1]-1); art_cols.append(A_std.shape[1]-1)
```

```
    else: # equality
```

```
        col = np.zeros((m,1)); col[i,0] = 1.0 # artificial
```

```
        A_std = np.hstack((A_std, col)); names.append(f"a{i+1}")
```

```
        basis.append(A_std.shape[1]-1); art_cols.append(A_std.shape[1]-1)
```

```
# Phase-I objective: maximize -(sum artificials) (i.e., minimize their sum)
```

```
c1 = np.zeros(A_std.shape[1]); [c1.__setitem__(j, -1.0) for j in art_cols]
```

```
return A_std, b_std, names, basis, art_cols, c1
```

```
def build_tableau(A_std, b_std, cost, basis):
```

```
    """
```

```
    Build the (m+1) x (n+1) tableau with top '-Z' row holding reduced costs.
```

```
    Start with row0 = cost and subtract c_B times each basic row to get
```

```
    reduced costs (c_j - z_j). RHS of row 0 equals (-Z).
```

```
    """
```

```
    m, n = A_std.shape
```

```
    T = np.zeros((m+1, n+1))
```

```
    T[1:, :n] = A_std; T[1:, -1] = b_std
```

```
    T[0, :n] = cost; T[0, -1] = 0.0
```

```
    cB = T[0, basis]
```

```
    for i, bcol in enumerate(basis, start=1):
```

```
        T[0, :] -= cB[i-1]*T[i, :]
```

```
    return T
```

```
def choose_entering(T):
```

```
    """Dantzig rule (maximization): choose the column with largest positive reduced cost."""
```

```
    reduced = T[0, :-1]
```

```
    idx = np.where(reduced > 1e-10)[0]
```

```
    return int(idx[np.argmax(reduced[idx])]) if len(idx) else None
```

```
def ratio_test(T, j):
```

```
    """Minimum ratio among rows with a_ij>0; returns pivot row index. None → unbounded."""
```

```
    rows, ratios = [], []
```

```
    for i in range(1, T.shape[0]):
```

```
        a = T[i, j]
```

```
        if a > 1e-12:
```

```
            rows.append(i); ratios.append(T[i, -1] / a)
```

```
    if not ratios: return None
```

```
    return rows[int(np.argmin(ratios))]
```

```
def pivot(T, i, j):
```

```
    """Gauss–Jordan pivot: normalize pivot row and zero out column j elsewhere."""
```

```
    piv = T[i, j]; T[i, :] /= piv
```

```
for r in range(T.shape[0]):
```

```
    if r == i: continue
```

```
    T[r, :] -= T[r, j] * T[i, :]
```

```
def print_tableau(T, basis, names, note=""):
```

```
    """
```

Pretty print tableau with beginner explanations.

Top row '-Z' shows reduced costs ($c_j - z_j$). Positive $\rightarrow$ entering candidate.

RHS of '-Z' equals $(-Z)$. At optimum, that equals $-Z^*$. Row labels are BASIC vars.

```
    """
```

```
df = pd.DataFrame(T, index=["-Z"] + [names[j] for j in basis],
```

```
                    columns=names + ["RHS"])
```

```
print("\n" + "="*120)
```

```
print(f"SIMPLEX TABLEAU {note}")
```

```
print("- Row '-Z': reduced costs ( $c_j - z_j$ ). Positive  $\rightarrow$  ENTER to improve Z (max).")
```

```
print("- RHS in '-Z' equals  $(-Z)$ . At optimum this equals  $-Z^*$ . Row labels are BASIC vars.")
```

```
print("="*120)
```

```
print(df.to_string())
```

```
return df
```

```
def simplex_loop(T, basis, names, phase_tag="Phase"):
```

```
    """
```

Run simplex with:

- prints of enter/leave and the full tableau every iteration,
- path of BFS snapshots for the SAG.

Returns final tableau, final basis, and the path.

```
    """
```

```
path = []
```

```
def read_solution():
```

```
    x = np.zeros(T.shape[1]-1)
```

```
    for r, bcol in enumerate(basis, start=1): x[bcol] = T[r, -1]
```

```
    return x, -T[0, -1]
```

```
# initial
```

```
print_tableau(T, basis, names, note=f"({phase_tag} start: basis {[names[j] for j in basis]})")
```

```
x0, z0 = read_solution(); path.append({"basis": basis.copy(), "x": x0.copy(), "z": z0})
```

```
it = 0
```

```
while True:
```

```
    it += 1
```

```

    j = choose_entering(T)
    if j is None: break
    i = ratio_test(T, j)
    if i is None: raise RuntimeError(f"Unbounded in {phase_tag}")
    print(f"\nIteration {it} ({phase_tag}): ENTER {names[j]}, LEAVE {names[basis[i-1]]}")
    pivot(T, i, j); basis[i-1] = j
    print_tableau(T, basis, names, note=f"({phase_tag} after pivot {it})")
    x, z = read_solution(); path.append({"basis":basis.copy(), "x":x.copy(), "z":z})
    return T, basis, path

# Phase I
t_phase1 = time.perf_counter()
A_std, b_std, names_all, B0, art_cols, c1 = make_standard_form(A, b, sense_list, var_names)
T1 = build_tableau(A_std, b_std, c1, B0.copy())
T1f, B1f, path1 = simplex_loop(T1, B0.copy(), names_all, "Phase I")
if abs(-T1f[0,-1]) > 1e-8:
    raise RuntimeError("Infeasible: artificials > 0 at Phase-I optimum.")
print_perf("Phase I (two-phase simplex)", t_phase1)

# Phase II: drop artificials; rebuild cost row for the *true* objective c
t_phase2 = time.perf_counter()
keep = np.ones(T1f.shape[1]-1, dtype=bool); [keep.__setitem__(j, False) for j in art_cols]
T2b = np.zeros((T1f.shape[0], keep.sum()+1))
T2b[1:, :-1] = T1f[1:, :-1][:, keep]; T2b[1:, -1] = T1f[1:, -1]
names2 = [names_all[j] for j in range(len(keep)) if keep[j]]

# true objective (only original decision vars carry coefficients)
c2 = np.zeros(len(names2))
for j, nm in enumerate(names2):
    if nm in var_names: c2[j] = c[var_names.index(nm)]

# Map Phase-I basis into reduced columns (replace artificials if they were basic)
B2 = []
for j in B1f:
    if j in art_cols:
        # Find the row where that artificial was basic and pivot any non-artificial in
        row = B1f.index(j) + 1
        cand = np.where(abs(T2b[row, :-1]) > 1e-9)[0]

```

```

enter = int(cand[0])
piv = T2b[row, enter]
T2b[row, :] /= piv
for r in range(T2b.shape[0]):
    if r == row: continue
    T2b[r, :] -= T2b[r, enter]*T2b[row, :]
B2.append(enter)
else:
    new_idx = np.where(keep[:,1])[0].tolist().index(j)
    B2.append(new_idx)

# Build Phase-II reduced-cost row and run simplex
T2 = T2b.copy(); T2[0,:] = 0.0; T2[0,-1] = c2
cB = T2[0, B2]
for r, bcol in enumerate(B2, start=1): T2[0, :] -= cB[r-1]*T2[r, :]
T2f, Bf, path2 = simplex_loop(T2, B2.copy(), names2, "Phase II")
print_perf("Phase II (two-phase simplex)", t_phase2)

# Read final solution (over current columns)
x_all = np.zeros(T2f.shape[1]-1)
for r, bcol in enumerate(Bf, start=1): x_all[bcol] = T2f[r,-1]
Z_star = -T2f[0,-1]

basis_names = [names2[j] for j in Bf]
nonbasic_names = [nm for nm in names2 if nm not in basis_names]
print("\n=== FINAL (two-phase tableau) ===")
print("Z* =", Z_star)
print("BASIC:", basis_names)
print("NONBASIC:", nonbasic_names)
print("Decision vars:", {nm: x_all[names2.index(nm)] if nm in names2 else 0.0 for nm in var_names})

# -----
# (3) SAG — Simplex adjacency graph (saved + shown)
# -----
def build_sag_table(path_phase1, names_phase1, path_phase2, names_phase2):
    """Combine Phase-I and Phase-II BFS snapshots into one readable table."""
    rows=[]; node=0
    def add(path, names, tag):

```

```

nonlocal node
for snap in path:
    bi = snap["basis"]; xv = snap["x"]; z = snap["z"]
    rec = {"node":node, "phase":tag, "Z":z,
          "BI_idx":{"+", ".join(str(j+1) for j in bi)+"},
          "BI_names":{"+", ".join(names[j] for j in bi)+"}}
    for v in var_names:
        rec[v] = xv[names.index(v)] if v in names else 0.0
    rows.append(rec); node += 1
add(path_phase1, names_phase1, "I")
add(path_phase2, names_phase2, "II")
return pd.DataFrame(rows)

```

```

def plot_adjacency_readable(path_phase1, names_phase1,
                           path_phase2, names_phase2,
                           nodes_per_row=4, node_size=1800, font_size=9,
                           filename_png="SAG_full.png", filename_pdf="SAG_full.pdf",
                           show_plot=True):

```

```

"""

```

Readable Simplex Adjacency Graph (SAG). Saves to ABSOLUTE paths in your current working folder and also shows a window (if backend supports it).

Each node is a BFS; edges correspond to one pivot operation.

```

"""

```

```

combined = [(snap, names_phase1, "I") for snap in path_phase1] + \
            [(snap, names_phase2, "II") for snap in path_phase2]
n = len(combined); rows = math.ceil(n / nodes_per_row)
xs, ys = [], []
for k in range(n):
    r = k // nodes_per_row; c = k % nodes_per_row
    xs.append(c); ys.append(-r)

```

```

fig = plt.figure(figsize=(min(22, 4 + 3.2*nodes_per_row), 4 + 2.4*rows))

```

```

for i, (snap, names_here, tag) in enumerate(combined):
    bi = snap["basis"]; z = snap["z"]; xv = snap["x"]
    bi_idx = "{" + ", ".join(str(j+1) for j in bi) + "}"
    bi_names = "{" + ", ".join(names_here[j] for j in bi) + "}"
    lines=[]
    for grp in [("x1", "x2", "x3"), ("x4", "x5", "x6"), ("x7", "x8", "x9")]:

```

```

        lines.append(" ".join(f"{v}={xv[names_here.index(v)].:.3g}" if v in names_here else f"{v}=—"
                                for v in grp))

    label = f"Node {i} (Phase {tag})\nBI={bi_idx}\nB={bi_names}\nZ={z:.3g}\n" + "\n".join(lines)

    plt.scatter(xs[i], ys[i], s=node_size, alpha=0.55 if i < n-1 else 0.75,
                color="#4c78a8", edgecolor="k")

    plt.annotate(label, (xs[i], ys[i]), ha="center", va="center", fontsize=font_size)

    if i < n-1:

        plt.annotate("", xy=(xs[i+1], ys[i+1]), xytext=(xs[i], ys[i]),
                    arrowprops=dict(arrowstyle="->", color="gray"))

plt.axis("off")

plt.title("Simplex adjacency path (node=BFS; edge=one pivot)", fontsize=12)

plt.tight_layout()

# Save to absolute paths in the current working directory
cwd = os.getcwd()
out_png = os.path.abspath(os.path.join(cwd, filename_png))
out_pdf = os.path.abspath(os.path.join(cwd, filename_pdf))
fig.savefig(out_png, dpi=300, bbox_inches="tight")
fig.savefig(out_pdf, dpi=300, bbox_inches="tight")
print(f"[SAG] Saved PNG → {out_png}")
print(f"[SAG] Saved PDF → {out_pdf}")

if show_plot:
    plt.show()
else:
    plt.close(fig)

t_sag = time.perf_counter()
nodes_df = build_sag_table(path1, names_all, path2, names2)
print("\nSAG nodes (BI and all x1..x9):")
print(nodes_df.to_string(index=False))
plot_adjacency_readable(path1, names_all, path2, names2,
                        nodes_per_row=4, node_size=1800, font_size=9,
                        filename_png="SAG_full.png", filename_pdf="SAG_full.pdf",
                        show_plot=True)
print_perf("SAG (build + plot + save)", t_sag)

```

```

# -----
# (4) Build B, B^{-1}, N, c_B, c_N and REDUCED COSTS
# -----

t_mats = time.perf_counter()

# Extract canonical constraint matrix from the final tableau

A_can = T2f[1:, :-1]      # size m_eq x n_reduced (columns = names2)
b_eq = T2f[1:, -1]
m_eq = A_can.shape[0]

B_cols = Bf
N_cols = [j for j in range(A_can.shape[1]) if j not in B_cols]
B_mat = A_can[:, B_cols]
N_mat = A_can[:, N_cols]
B_inv = np.linalg.inv(B_mat)

# Cost vector across current columns; decision vars carry original weights
c_full = np.zeros(A_can.shape[1])
for j, nm in enumerate(names2):
    if nm in var_names: c_full[j] = c[var_names.index(nm)]
cB = c_full[B_cols]
cN = c_full[N_cols]

# Reduced costs of non-basics:  $\bar{r} = c_N - c_B^T B^{-1} N$ 
reduced = cN - cB @ B_inv @ N_mat

xB = b_eq.copy()      # basic variable values (since B is canonical)
yT = cB @ B_inv      # dual prices (row vector y^T)

print("\n--- MATRICES ---")
print("B shape:", B_mat.shape, " N shape:", N_mat.shape)
print("Max |B - I| (0 at canonical BFS) =", float(np.max(np.abs(B_mat - np.eye(m_eq)))))
print("Reduced costs (non-basics):", reduced)
print_perf("Matrices & reduced costs", t_mats)

# -----
# (5) SENSITIVITY ANALYSIS — EXACT SLIDE FORMULAS (x1..x9 ONLY)
# -----

t_sa = time.perf_counter()

# We **only** analyze  $\alpha \in \{x1,..,x9\}$  (decision variables) as requested.

```

```

# • If  $x_\alpha$  is BASIC in the final basis  $\rightarrow$  (A): compute  $\delta$ -interval using ONLY
# the nonbasic **decision-variable** columns in  $N$ .

# • If  $x_\alpha$  is NON-BASIC  $\rightarrow$  (B):  $\delta \leq -\bar{r}_\alpha$  using that variable's reduced cost only.

# • RHS (C) remains standard: feasibility with  $B^{-1}b + \gamma B^{-1}e_i \geq 0$ .

# Map each decision var to its column index in the final (Phase-II) tableau columns
dec2col = {nm: names2.index(nm) for nm in var_names if nm in names2}

# Identify which nonbasic columns correspond to decision variables (exclude  $s_i, u_i, a_i$ )
N_dec_local_idx = [] # indices into reduced[] / N_mat columns for decision variables only
N_dec_cols = [] # the actual column indices in the tableau (names2-space)
N_dec_names = [] # their names ("x1"..)
for idx_local, jcol in enumerate(N_cols):
    nm = names2[jcol]
    if nm in var_names:
        N_dec_local_idx.append(idx_local)
        N_dec_cols.append(jcol)
        N_dec_names.append(nm)

# Quick helper to fetch reduced cost for a given **decision** nonbasic column
rc_map = {}
for idx_local, jcol in enumerate(N_cols):
    nm = names2[jcol]
    if nm in var_names:
        rc_map[nm] = reduced[idx_local]

print("\nSENSITIVITY ANALYSIS — Objective coefficients (ONLY decision vars  $\alpha = x_1..x_9$ )")
print(" (A) If  $\alpha$  is BASIC: use  $(c_N^T - (c_B + \delta e_k)^T B^{-1}N)_\alpha \leq 0$  over NONBASIC decision columns.")
print(" (B) If  $\alpha$  is NON-BASIC:  $\bar{r}_\alpha(\delta) = \bar{r}_\alpha + \delta \leq 0 \Rightarrow \delta \leq -\bar{r}_\alpha$ .")

# Pre-compute  $B^{-1}N$  and then select only the decision-variable columns of  $N$ 
BinVN = B_inv @ N_mat
BinVN_dec = BinVN[:, N_dec_local_idx] #  $m_{eq} \times (\# \text{ nonbasic decision vars})$ 
reduced_dec = reduced[N_dec_local_idx] # reduced costs aligned with decision-only  $N$  columns

for alpha, nm in enumerate(var_names, start=1): #  $\alpha = 1..9$  in  $x_1..x_9$  order
    if nm not in dec2col:
        print(f" {nm:>2s}: not present among current columns (unexpected).")

```

```

    continue

col = dec2col[nm]
if col in B_cols:
    # BASIC decision variable  $\rightarrow (A)$ .

    k = B_cols.index(col) # row position in B

    v_dec = BinvN_dec[k, :] # row k restricted to decision-only nonbasic columns

    lows, ups = [], []

    for rbar, vij in zip(reduced_dec, v_dec):
        if vij > 1e-12: lows.append(rbar / vij) #  $\delta \geq \bar{r} / v$ 
        elif vij < -1e-12: ups.append(rbar / vij) #  $\delta \leq \bar{r} / v$ 
        # if vij  $\approx 0$ : no restriction from that column

    lo = max(lows) if lows else -math.inf
    up = min(ups) if ups else math.inf

    # Objective change:  $\Delta Z = \delta \cdot xB\_k$  ( $xB\_k$  is the basic value of that variable)

    print(f" {nm:>2s} (BASIC,  $\alpha=\{\alpha\}$ ):  $\delta \in [{lo:.6g}, {up:.6g}]$  and  $\Delta Z = {xB[k]:.6g} \cdot \delta$ ")
else:
    # NON-BASIC decision variable  $\rightarrow (B)$ 

    rbar = rc_map[nm] # reduced cost for THIS decision nonbasic variable only

    print(f" {nm:>2s} (NON-BASIC,  $\alpha=\{\alpha\}$ ): allowable  $\delta \leq {(-rbar):.6g}$  (since  $\bar{r}_\alpha + \delta \leq 0$ )")

print("\nSENSITIVITY ANALYSIS — RHS changes (C):  $b\_i \leftarrow b\_i + \gamma$ ")
print(" Feasibility requires  $B^{-1}b + \gamma B^{-1} e\_i \geq 0$ . Let d = column i of  $B^{-1}$ .")
print(" For each basic k:  $xB\_k + \gamma d\_k \geq 0 \Rightarrow$  bounds on  $\gamma$ .  $\Delta Z = \gamma \cdot \gamma\_i$ , where  $\gamma = c\_B^T B^{-1} \cdot$ .")

for i in range(m_eq):
    d = B_inv[:, i] # column i of  $B^{-1}$ 

    lows, ups = [], []

    for k in range(m_eq):
        if d[k] > 1e-12: ups.append(xB[k] / d[k]) #  $\gamma \leq xB\_k / d\_k$ 
        elif d[k] < -1e-12: lows.append(-xB[k] / d[k]) #  $\gamma \geq -xB\_k / d\_k$ 

    lo = max(lows) if lows else -math.inf
    up = min(ups) if ups else math.inf

    print(f" constraint row {i+1:>2d}:  $\gamma \in [{lo:.6g}, {up:.6g}]$  and  $\Delta Z = {\gamma T[i]:.6g} \cdot \gamma$ ")

# Degeneracy check (any basic variable value  $\sim 0$ ?)

print("\nDegenerate optimum?  $\rightarrow$ ", any(abs(val) < 1e-12 for val in xB))

print_perf("Sensitivity analysis", t_sa)

```

```

# -----
# (6) Solve the DUAL directly (PuLP/CBC) and print y
# -----

t_dual = time.perf_counter()

try:

    import pulp

    dual = pulp.LpProblem("WorkoutLP_dual", pulp.LpMinimize)

    y=[]

    # Dual variable signs depend on primal constraint sense:
    # primal  $\leq \rightarrow y \geq 0$    primal  $\geq \rightarrow y \leq 0$    primal  $= \rightarrow y$  free

    for i in range(m):

        if sense_list[i] == "<=": y.append(pulp.LpVariable(f"y{i+1}", lowBound=0))

        elif sense_list[i] == ">=": y.append(pulp.LpVariable(f"y{i+1}", upBound=0))

        else:

            yp = pulp.LpVariable(f"y{i+1}_p", lowBound=0)

            ym = pulp.LpVariable(f"y{i+1}_m", lowBound=0)

            y.append(yp - ym) # free

    dual += pulp.lpSum(b[i]*y[i] for i in range(m)) # minimize  $b^T y$ 

    # Dual constraints:  $A^T y \geq c$  (because primal is max with  $x \geq 0$ )

    for j in range(n):

        dual += pulp.lpSum(A[i,j]*y[i] for i in range(m)) >= c[j], f"dual_col_{var_names[j]}"

    dual.solve(pulp.PULP_CBC_CMD(msg=False))

    print("\nDUAL status:", pulp.LpStatus[dual.status])

    print("DUAL objective  $b^T y =$ ", pulp.value(dual.objective), " (should equal  $Z^*$ )")

    for i in range(m):

        yi = pulp.value(y[i])

        sign = ">=0" if sense_list[i]=="<=" else "<=0" if sense_list[i]==">=" else "free"

        print(f" y[{i+1:>2d}] ({sign:>4s}) = {yi:.6g}")

    print_perf("Dual (PuLP/CBC)", t_dual)

except Exception as e:

    print("Dual solve failed:", e)

    print_perf("Dual attempt (failed)", t_dual)

# =====
# (PERF) Total script runtime + final memory snapshot — ADDED PER REQUEST
# =====

print_perf("TOTAL script runtime", _SCRIPT_T0)

```

2^ο πρόβλημα

```
# =====  
# CIRCUIT TRAINING — MILP (PuLP/CBC) with ONE HAMILTONIAN PATH  
# =====  
# WHAT THIS SCRIPT DOES (high level):  
# • Builds a Mixed-Integer Linear Program (MILP) that picks an ordered circuit  
#   of 27 exercises (ALL of them) subject to muscle-usage and intensity rules.  
# • Enforces a SINGLE directed Hamiltonian path that visits each exercise once.  
# • Minimizes total time = sum(durations) + sum(transition times).  
# • Solves with CBC via PuLP, then reconstructs and prints the ordered path.  
#  
# MODELING INSIGHT (paper form; no explicit start/end binaries):  
# VARIABLES:  
# • included[i] ∈ {0,1}   (x_i in the paper): whether exercise i is selected  
# • next[i->j] ∈ {0,1}   (f_ij in the paper): arc i→j is used (i≠j)  
# • order[i] ∈ [0, N]    (s_i in the paper): position index for MTZ  
#  
# CORE CONSTRAINTS (paper equations numbering in comments):  
# (2.1)  $\sum \text{included} = N$   
# (2.2)  $\sum \text{next} = N - 1$   
# (2.3)  $\text{next}[i \rightarrow j] \leq \text{included}[i]$   
# (2.4)  $\text{next}[i \rightarrow j] \leq \text{included}[j]$   
# (2.5)  $\sum_j \text{next}[i \rightarrow j] \leq 1$    (at most one successor per node)  
# (2.6)  $\sum_i \text{next}[i \rightarrow j] \leq 1$    (at most one predecessor per node)  
# (2.7) MTZ (no cycles):  $\text{order}[i] - \text{order}[j] + (N+1) * \text{next}[i \rightarrow j] \leq N, \forall i \neq j$   
# (2.9)  $\text{order}[i] \leq N * (\text{outdeg}(i) + \text{indeg}(i))$  (order=0 if i has no incident arc)  
# (2.10)  $\sum \text{order} = N(N+1)/2$   
#  
# DOMAIN RULES (paper-style; same spirit as your original code):  
# • No two primaries in a row for the same muscle (forbid those arcs).  
# • No "three in a row" using the same muscle (pairwise  $\leq 1$  on consecutive arcs).  
# • Muscle totals (how many selected hit a muscle) within [lower, upper].  
# • No two high-intensity back-to-back (forbid those arcs).
```

```

# • Average MET band for the selected set.
#
# EXTRA PRINTS (for sanity / report):
# • A small "PARAMETERS" digest (data summary).
# • The number of decision variables (columns) and constraints (rows) we BUILT.
# • A derivation-by-formula "expected constraints" count (so you see where 5710 comes from).
# =====

from __future__ import annotations
import time
import numpy as np
import pandas as pd

# -----
# OPTIONAL: memory reporting (handy but not essential). If psutil missing, skip.
# -----

try:
    import psutil
    HAVE_PSUTIL = True
except Exception:
    HAVE_PSUTIL = False

def rss_MB():
    """Return current resident memory (MB) if psutil is available; else None."""
    return psutil.Process().memory_info().rss/1024**2 if HAVE_PSUTIL else None

# -----
# SOLVER IMPORT (PuLP gives free access to the CBC MILP solver)
# -----

try:
    import pulp # pip install pulp
except ImportError as e:
    raise SystemExit(
        "PuLP is not installed. Install it first:\n"
        " pip install pulp\n"
        "Then run the script with:\n"
        " py circuit_milp.py"
    )

```

```

# -----

# VERBOSITY & DEBUG TOGGLES

# -----

SHOW_SOLVER_LOG = True    # Set False to suppress CBC's console chatter

DEBUG_DEGREE_REPORT = False # Set True to print per-node in/out degrees after solve

DEBUG_PRINT_ARCS = False # Set True to print raw arc list (i -> j); normally OFF


# =====

# (1) DATA — the 27-exercise, 10-muscle instance

# =====

# EXERCISE SET (nodes of the path)

E = [

    "bench_press", "chest_fly", "dips", "push_up",

    "pull_up", "shoulder_press", "upright_row", "bent_over_row", "lat_pulldown", "biceps_curl",

    "deadlift", "squat", "back_extension", "crunch", "exercise_wheel", "side_plank", "leg_press",

    "lunge", "push_up_rotation", "jumping_jack", "jumping_squat", "burpee", "mountain_climber",

    "plyo_lunge", "lunge_jump", "step_up", "battle_rope"

]


# MUSCLE GROUPS (used for load balancing rules)

M = ["chest", "shoulders", "triceps", "biceps", "lats", "trapezius",

     "lower_back", "abs", "quads", "glutes_hams"]


# We are selecting EVERY exercise (you asked for all 27)

N_chosen = 27


# -----

# Duration per exercise (minutes)

# -----

duration = {

    "bench_press":0.42, "chest_fly":0.40, "dips":0.42, "push_up":0.40,

    "pull_up":0.40, "shoulder_press":0.40, "upright_row":0.40, "bent_over_row":0.42, "lat_pulldown":0.40, "biceps_curl":0.38,

    "deadlift":0.45, "squat":0.42, "back_extension":0.40, "crunch":0.35, "exercise_wheel":0.33, "side_plank":0.33, "leg_press":0.40,

    "lunge":0.40, "push_up_rotation":0.40, "jumping_jack":0.35, "jumping_squat":0.40, "burpee":0.45, "mountain_climber":0.38,

    "plyo_lunge":0.40, "lunge_jump":0.38, "step_up":0.40, "battle_rope":0.40

}

```

```

# -----
# Transition times  $t_{i,j}$  (minutes)
# -----

# Defaults to ~0.12 min ( $\approx 7.2$  seconds); some pairs longer to reflect station changes.
t = pd.DataFrame(0.12, index=E, columns=E)

for e in E:
    t.loc[e, e] = 0.0 # no self-arc

# Specific longer transitions (hand-tuned)
for (i, j), val in {
    ("exercise_wheel", "jumping_squat"):0.25,
    ("jumping_squat", "exercise_wheel"):0.25,
    ("upright_row", "biceps_curl"):0.20,
    ("deadlift", "squat"):0.20,
    ("squat", "deadlift"):0.20,
    ("bench_press", "chest_fly"):0.18,
    ("chest_fly", "bench_press"):0.18,
    ("pull_up", "lat_pulldown"):0.22,
    ("lat_pulldown", "pull_up"):0.22,
    ("battle_ropes", "biceps_curl"):0.20,
    ("bent_over_row", "deadlift"):0.22,
    ("deadlift", "bent_over_row"):0.22,
}.items():
    t.loc[i, j] = val

# -----
# Intensity (MET) data
# -----

MET = {
    "bench_press":7.0, "chest_fly":6.5, "dips":8.0, "push_up":7.0,
    "pull_up":8.0, "shoulder_press":7.5, "upright_row":6.5, "bent_over_row":6.8, "lat_pulldown":6.8, "biceps_curl":6.5,
    "deadlift":8.0, "squat":8.0, "back_extension":6.0, "crunch":6.0, "exercise_wheel":7.5, "side_plank":6.0, "leg_press":7.0,
    "lunge":7.5, "push_up_rotation":7.5, "jumping_jack":7.0, "jumping_squat":8.5, "burpee":9.5, "mountain_climber":8.0,
    "plyo_lunge":8.5, "lunge_jump":8.0, "step_up":7.0, "battle_ropes":8.5
}

T_MET = 8.0

high = {e: 1 if MET[e] >= T_MET else 0 for e in E} # 1 if high intensity

```

```

# -----
# Muscle work matrix  $w_{im} \in \{0,1,2\}$ 
# -----

# 0 = unused, 1 = secondary work, 2 = primary work on muscle  $m$  by exercise  $e$ .

work = pd.DataFrame(0, index=E, columns=M)

# Chest family

work.loc["bench_press", ["chest", "triceps", "shoulders"]] = [2,1,1]
work.loc["chest_fly", ["chest", "shoulders"]] = [2,1]
work.loc["dips", ["chest", "triceps"]] = [2,2]
work.loc["push_up", ["chest", "triceps", "abs"]] = [2,1,1]
work.loc["push_up_rotation", ["chest", "shoulders", "triceps", "abs"]] = [2,1,1,1]

# Back / pulls

work.loc["pull_up", ["lats", "biceps"]] = [2,1]
work.loc["bent_over_row", ["lats", "trapezius", "biceps", "lower_back"]] = [2,1,1,1]
work.loc["lat_pulldown", ["lats", "biceps"]] = [2,1]

# Shoulders

work.loc["shoulder_press", ["shoulders", "triceps"]] = [2,1]
work.loc["upright_row", ["shoulders", "trapezius"]] = [2,1]
work.loc["battle_rope", ["shoulders", "trapezius", "abs", "biceps", "triceps"]] = [2,1,1,1,1]

# Arms

work.loc["biceps_curl", ["biceps"]] = [2]

# Posterior chain / legs / core

work.loc["deadlift", ["lower_back", "glutes_hams", "quads", "trapezius"]] = [2,2,1,1]
work.loc["squat", ["quads", "glutes_hams"]] = [2,2]
work.loc["back_extension", ["lower_back", "glutes_hams"]] = [2,1]
work.loc["crunch", ["abs"]] = [2]
work.loc["exercise_wheel", ["abs", "lower_back"]] = [2,1]
work.loc["side_plank", ["abs"]] = [2]
work.loc["leg_press", ["quads", "glutes_hams"]] = [2,1]
work.loc["lunge", ["quads", "glutes_hams"]] = [2,2]
work.loc["jumping_jack", ["quads", "glutes_hams", "shoulders"]] = [1,1,1]
work.loc["jumping_squat", ["quads", "glutes_hams"]] = [2,2]
work.loc["burpee", ["quads", "glutes_hams", "chest", "shoulders", "triceps", "abs"]] = [2,2,1,1,1,1]

```

```

work.loc["mountain_climber", ["abs", "quads", "shoulders"]] = [2,1,1]
work.loc["plyo_lunge",      ["quads", "glutes_hams"]] = [2,2]
work.loc["lunge_jump",      ["quads", "glutes_hams"]] = [2,2]
work.loc["step_up",         ["quads", "glutes_hams"]] = [2,1]

# Binary mask "used" = 1{ w_im > 0 }
used = (work > 0).astype(int)

# -----
# Muscle totals (global load bounds)
# -----
muscle_lower_limit = {"chest":3, "shoulders":3, "triceps":3, "biceps":3, "lats":1, "trapezius":0,
                      "lower_back":3, "abs":5, "quads":5, "glutes_hams":5}
muscle_upper_limit = {"chest":20, "shoulders":20, "triceps":20, "biceps":12, "lats":10, "trapezius":12,
                      "lower_back":12, "abs":16, "quads":24, "glutes_hams":24}

# -----
# Average MET band constraints
# -----
intensity_lower_limit = 6.5
intensity_upper_limit = 9.2

# =====
# (2) BUILD AND SOLVE THE MILP (single Hamiltonian path; paper-only equations)
# =====
t0 = time.perf_counter()
print(f"{{{time.perf_counter()-t0}:5.2f}s | RSS~{{{rss_MB() or 0}:.1f} MB} Data ready.")
print("PARAMETERS")
print(f" |E|={len(E)} exercises, |M|={len(M)} muscles, N={N_chosen}")
print(f" mean(duration)={np.mean(list(duration.values())):.3f} min")
print(f" mean(MET)={np.mean(list(MET.values())):.2f} (high if >= {T_MET})")
print(f" mean(transition>0)={t.values[t.values>0].mean():.3f} min\n")

# -----
# Create PuLP problem: MINIMIZE total time (durations + transitions)
# -----
prob = pulp.LpProblem("CircuitTraining", pulp.LpMinimize)

```

```

# -----
# Decision variables (paper notation:  $x_i, f_{ij}, s_i$ )
# -----

included = {e: pulp.LpVariable(f"included[{e}]", 0, 1, cat="Binary") for e in E} #  $x_i$ 
next_ex = {(i, j): pulp.LpVariable(f"next[{i}->{j}]", 0, 1, cat="Binary")}

    for i in E for j in E if i != j:                #  $f_{ij}$ 
order = {e: pulp.LpVariable(f"order[{e}]", lowBound=0, upBound=N_chosen,
    cat="Continuous") for e in E}                #  $s_i$ 

# -----
# Objective function
# -----

prob += (
    pulp.lpSum(duration[e] * included[e] for e in E) +
    pulp.lpSum(t.loc[i, j] * next_ex[(i, j)] for i in E for j in E if i != j)
), "total_time"

# -----
# CORE: Paper's equations (2.1)–(2.10)
# -----

# (2.1) pick exactly N exercises
prob += pulp.lpSum(included[e] for e in E) == N_chosen, "number_of_exercises_in_circuit"

# (2.2) total arcs equals N-1
prob += pulp.lpSum(next_ex[(i, j)] for i in E for j in E if i != j) == N_chosen - 1, \
    "number_of_following_exercises_in_circuit"

# (2.3)–(2.4) link  $f_{ij}$  to  $x_i, x_j$ 
for i in E:
    for j in E:
        if i == j:
            continue

        prob += next_ex[(i, j)] <= included[i], f"follow_only_if_included_i[{i}->{j}]"
        prob += next_ex[(i, j)] <= included[j], f"follow_only_if_included_j[{i}->{j}]"

# (2.5) at most one outgoing per node
for i in E:

```

```

prob += pulp.lpSum(next_ex[(i, j)] for j in E if j != i) <= 1, f"at_most_one_follows[{i}]"

# (2.6) at most one incoming per node
for j in E:
    prob += pulp.lpSum(next_ex[(i, j)] for i in E if i != j) <= 1, f"at_most_one_precedes[{j}]"

# (2.7) MTZ / no-cycle; BIG = N+1, RHS = N (paper form)
BIG = N_chosen + 1
for i in E:
    for j in E:
        if i == j:
            continue
        prob += order[i] - order[j] + BIG * next_ex[(i, j)] <= N_chosen, f"no_cycle[{i}->{j}]"

# (2.9) dummy-zero-if-isolated: if node i has no incident arc, force order[i]=0
for i in E:
    prob += order[i] <= N_chosen * (
        pulp.lpSum(next_ex[(i, j)] for j in E if j != i) +
        pulp.lpSum(next_ex[(j, i)] for j in E if j != i)
    ), f"dummy_zero_if_not_included[{i}]"

# (2.10) sum of orders equals N(N+1)/2 (clean scaling; uniqueness of 1..N)
prob += pulp.lpSum(order[e] for e in E) == N_chosen * (N_chosen + 1) / 2, "sum_of_orders"

# -----
# DOMAIN-SPECIFIC RULES (paper-style; same spirit as your old code)
# -----

# (A) No two primaries in a row for the same muscle m: forbid those arcs
for m in M:
    prim = [e for e in E if work.loc[e, m] == 2]
    for i in prim:
        for j in prim:
            if i != j:
                prob += next_ex[(i, j)] == 0, f"no_two primaries[{m},{i}->{j}]"

# (B) No "three in a row" using the same muscle m:
for m in M:

```

```

used_m = [e for e in E if used.loc[e, m] == 1]

for i in used_m:
    for j in used_m:
        if i == j:
            continue
        for k in used_m:
            if k == j or k == i:
                continue
            prob += next_ex[(i, j)] + next_ex[(j, k)] <= 1, f"no_three_in_row[{m},{i}->{j}->{k}]"

# (C) Muscle totals (global load bounds): how many selected exercises hit muscle m
for m in M:
    prob += pulp.lpSum(used.loc[e, m] * included[e] for e in E) >= muscle_lower_limit[m], f"muscle_total_lb[{m}]"
    prob += pulp.lpSum(used.loc[e, m] * included[e] for e in E) <= muscle_upper_limit[m], f"muscle_total_ub[{m}]"

# (D) No two high-intensity exercises back-to-back:
for i in E:
    for j in E:
        if i != j and high[i] == 1 and high[j] == 1:
            prob += next_ex[(i, j)] == 0, f"no_two_high[{i}->{j}]"

# (E) Average MET over the selected set within a band:
prob += pulp.lpSum(MET[e] * included[e] for e in E) >= intensity_lower_limit * N_chosen, "intensity_lb"
prob += pulp.lpSum(MET[e] * included[e] for e in E) <= intensity_upper_limit * N_chosen, "intensity_ub"

# -----
# >>> NEW: PRINT MODEL SIZE (decision variables & constraints) BEFORE SOLVE
# -----

all_vars = prob.variables() # decision variables only (PuLP objects)
num_vars = len(all_vars)
num_rows = len(prob.constraints) # constraints we built (before presolve)

# Derive counts used in the "formula explanation" so you can see 5710:
N = N_chosen
pairs = N * (N - 1)

# Count primaries per muscle and "used" per muscle for the sums:
p_m_list = [int((work[m] == 2).sum()) for m in M]
a_m_list = [int(used[m].sum()) for m in M]

```

```

h = sum(high.values())

count_select_N = 1          # (2.1)
count_arcs_Nm1 = 1          # (2.2)
count_link_ij = 2 * pairs   # (2.3)+(2.4)
count_out_le1 = N           # (2.5)
count_in_le1 = N            # (2.6)
count_mtz = pairs           # (2.7)
count_iso0 = N              # (2.9)
count_sumorder = 1          # (2.10)
count_no2prim = sum(p * (p - 1) for p in p_m_list)
count_no3row = sum(a * (a - 1) * (a - 2) for a in a_m_list)
count_muscle_total = 2 * len(M)    # lower+upper per muscle
count_no_high = h * (h - 1)    # ordered pairs
count_metband = 2

expected_constraints = (
    count_select_N + count_arcs_Nm1 + count_link_ij + count_out_le1 + count_in_le1 +
    count_mtz + count_iso0 + count_sumorder + count_no2prim + count_no3row +
    count_muscle_total + count_no_high + count_metband
)

print("MODEL SIZE (pre-presolve)")
print(f" Decision variables (columns): {num_vars}")
print(f" Constraints (rows): {num_rows}")
print(" By-formula expected rows breakdown:")
print(f" (2.1) select N : {count_select_N}")
print(f" (2.2) arcs N-1 : {count_arcs_Nm1}")
print(f" (2.3)+(2.4) link f to x : {count_link_ij}")
print(f" (2.5) out-degree ≤ 1 : {count_out_le1}")
print(f" (2.6) in-degree ≤ 1 : {count_in_le1}")
print(f" (2.7) MTZ pairs : {count_mtz}")
print(f" (2.9) order zero if isolated : {count_iso0}")
print(f" (2.10) sum of orders : {count_sumorder}")
print(f" no two primaries in a row : {count_no2prim}")
print(f" no three-in-a-row per muscle : {count_no3row}")
print(f" muscle totals (lb+ub) : {count_muscle_total}")
print(f" no high→high : {count_no_high}")

```

```

print(f" MET band (lb+ub) : {count_metband}")
print(f" EXPECTED TOTAL : {expected_constraints}\n")

# -----
# SOLVE the MILP with CBC
# -----

solver = pulp.PULP_CBC_CMD(msg=SHOW_SOLVER_LOG)
t_solve = time.perf_counter()
prob.solve(solver)
t_solve = time.perf_counter() - t_solve

# =====
# (3) READ SOLUTION + RECONSTRUCT THE SINGLE ORDERED PATH
# =====

status = pulp.LpStatus[prob.status]
print("\n=== MILP (CBC) RESULT ===")
print("Status:", status)
obj_val = pulp.value(prob.objective)
print("Objective (minutes):", float(obj_val) if obj_val is not None else None)

if status != "Optimal":
    print("[WARN] Model status is not Optimal. The constraints may be too tight or conflicting.")
    print(f"[Solve time: {t_solve:.2f}s | RSS≈{(rss_MB() or 0):.1f} MB]")
    raise SystemExit(0)

# Selected set (should be all 27 by construction)
sel = [e for e in E if (included[e].value() or 0) > 0.5]
print(f"Selected count = {len(sel)} (expected {N_chosen})")

# >>> Print inclusion binaries x_i for all exercises
print("\nInclusion variables x[e] (included[e]):")
for e in E:
    val = included[e].value()
    x = int(round(val)) if val is not None else None
    print(f" x[{e}] = {x}")

# Build successor (succ) and predecessor (pred) maps from chosen arcs next[i->j]=1
succ, pred, arc_list = {}, {}, []

```

```

for i in E:
    for j in E:
        if i == j:
            continue

        val = next_ex[(i, j)].value()

        if val is not None and val > 0.5:
            succ[i] = j
            pred[j] = i
            arc_list.append((i, j))

if DEBUG_PRINT_ARCS:
    print("\n[DEBUG] Raw arcs with next=1 (each is an edge in the path):")
    for (i, j) in arc_list:
        print(f" {i} -> {j}")

print("\nArc count chosen:", len(arc_list), "(expected N-1 =", N_chosen-1, ")")

# Identify the true start node (no predecessor)
start_node = next((e for e in sel if e not in pred), None)

# Reconstruct the path by walking succ from the start node
path = []

if start_node is not None:
    cur = start_node
    seen = set()

    while cur is not None and cur not in seen:
        path.append(cur)
        seen.add(cur)
        cur = succ.get(cur, None)

# Sanity check: path length should be N (27). If not, print diagnostics.
if len(path) == N_chosen:
    print("\nPath (ordered):")
    print(" " + " -> ".join(path))
else:
    print("\n[WARN] Reconstructed path length =", len(path), f"(expected {N_chosen}).")
    if path:
        print("Partial path:")

```

```

    print(" " + " " -> ".join(path))

leftovers = [e for e in sel if e not in set(path)]

if leftovers:

    print("Unreached selected nodes:", leftovers)


# Optional: degree report per node
if DEBUG_DEGREE_REPORT:

    print("\n[DEBUG] Degree report (in, out) for selected nodes:")

    for e in sel:

        out_deg = sum(1 for j in E if j != e and (next_ex[(e, j)].value() or 0) > 0.5)

        in_deg = sum(1 for i in E if i != e and (next_ex[(i, e)].value() or 0) > 0.5)

        print(f" {e:18s} in={in_deg} out={out_deg}")


print(f"\n[Solve time: {t_solve:.2f}s | RSS≈{(rss_MB() or 0):.1f} MB]")

```

11.Βιβλιογραφία

- [1] [High-Intensity Interval Training: For Fitness, for Health or Both?](#)
- [2] [Plyometric Training](#)
- [3] [Search Results for “MET” – Compendium of Physical Activities](#)
- [4] <https://pmc.ncbi.nlm.nih.gov/articles/PMC11197041/>
- [5] [How to Measure Physical Activity Intensity | Physical Activity Basics | CDC](#)
- [6] <https://www.sciencedirect.com/science/article/pii/S0270025587904738>
- [7] <https://pmc.ncbi.nlm.nih.gov/articles/PMC5673663/>
- [8] <https://www.cdc.gov/physical-activity-basics/guidelines/adults.html>
- [9] <https://iris.who.int/bitstream/handle/10665/337001/9789240014886-eng.pdf>
- [10] https://www.nsga.com/education/articles/kinetic-select/plyometric-exercises/?srsltid=AfmBOopNSzUiCINUwgpQj7u2N4X5RxbR_YRrB89Wq2J4SmdAtGym4427&
- [11] <https://www.nsga.com/education/articles/kinetic-select/stretch-shortening-cycle/?srsltid=AfmBOooQYOvVBtd56PVGfhqstKMymj3MGTegw21EqjfJvssydQYZJbPO&>
- [12] [American College of Sports Medicine position stand. Quantity and quality of exercise for developing and maintaining cardiorespiratory, musculoskeletal, and neuromotor fitness in apparently healthy adults: guidance for prescribing exercise - PubMed](#)
- [13] [An Optimization Model for Exercise Sheduling](#)
- [14] [lecture10.pdf](#)
- [15] [Simplex.pdf](#)
- [16] https://www.dl.behinehyab.com/Ebooks/NETWORK/NET005_354338_www.behinehyab.com.pdf
- [17] <https://homepages.cwi.nl/~dadush/workshop/discrepancy-ip/papers/heuristics-survey-fischetti-lodi-11.pdf>
- [18] <https://arxiv.org/abs/1611.09940>

[19] <https://d-krupke.github.io/cpsat-primer/>